AMD

ROCm Certification

Level 1: An Introduction to AMD GPUs and ROCm for AI and HPC



Public

THEORY

Module 1

GPU INTRODUCTION

What a GPU is and why GPUs dominate AI/HPC.
Where AMD sits in the compute landscape and which workloads benefit the most.

AMD ROCm™ STACK

Ecosystem architecture, releases, and compatibility.
Map an AMD ROCm™ version to GPU, LLVM target, and supported OS.

HIP

What HIP is (not emulation) and how it maps to CUDA.
One source compiles for AMD or NVIDIA; hipify ports CUDA to HIP.

ROCm TOOLS

amd-smi, rocminfo, hipcc, rocprofv3 – the daily workflow
Commands cover monitor, query, compile, profile.

AMD
together we advance_

Public

Module 2

AMD INSTINCT

The Instinct lineage, the MI300 package down to one Compute Unit, and the on-chip memory hierarchy from registers to HBM.

GPU Programming and Execution Model

How your code maps to the chip: grid → block → wavefront → SIMD. The SIMT model, and how wavefronts hide memory latency.

CDNA vs RDNA

Two AMD GPU architectures — one for data-center compute, one for professional and consumer compute and graphics. Same HIP source compiles for both.

AMD
together we advance_

Public

THEORY

Module 3

ROCM COMPUTE LIBRARIES

Pre-tuned building blocks for GEMM, convolutions, FFTs, solvers, RNG, sparse, and parallel primitives — plus Triton for custom kernels.

PYTORCH ON ROCM

The standard PyTorch workflow, native on AMD — the training loop and torch.compile().

SCALING & SERVING

Beyond a single GPU — multi-GPU and multi-node training with RCCL and inference with PyTorch and vLLM.

AMD
together we advance_

Public

THEORY

Module 4

HIP Programming Concepts

HIP concepts to get started.
Reminder of HIP structure and thread hierarchy; host-device programming.

HIP Programming

Introduction to HIP programming.
Vector addition example; error checking; timing HIP operations.

Porting from CUDA to HIP

Translating CUDA source to HIP
Automatic translation tools; vector addition porting example.

AMD
together we advance_

Public

Module 5

DIAGNOSE BOTTLENECKS

Roofline + rocprofv3
Use roofline analysis to distinguish memory- from compute-limited kernels, then use rocprofv3 to measure kernel runtime, collect performance counters.

OPTIMIZE MEMORY ACCESS

Coalesce and tile
Use LDS tiling so the transpose performs coalesced global reads and writes — same work, more efficient memory access.

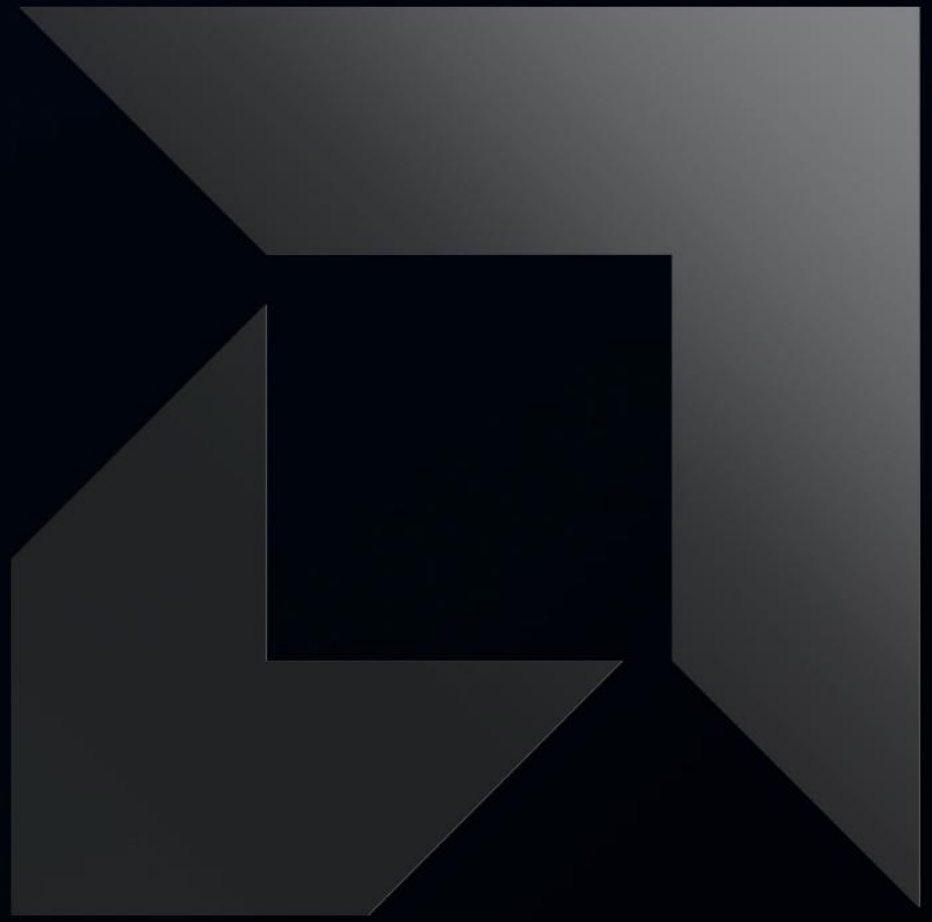
TUNE BLOCK SIZE

Block (workgroup) size
Sweep SAXPY block sizes and observe how available parallelism and resource use affect performance.

EVALUATE REDUCED PRECISION

FP16/BF16
Benchmark FP32, FP16, BF16 GEMMs; compare throughput and numerical difference relative to FP32, and explain the role of FP32 accumulation.

AMD
together we advance_



ROCm™ Certification – Level 1

An Introduction to AMD GPUs and ROCm for AI and HPC

Module 1: GPUs, ROCm, and HIP

AMD
together we advance_

Public

Module 1 Objectives

GPU INTRODUCTION

What a GPU is and why GPUs dominate AI/HPC.
Where AMD sits in the compute landscape and which workloads benefit the most.

AMD ROCm™ STACK

Ecosystem architecture, releases, and compatibility.
Map an AMD ROCm™ version to GPU, LLVM target, and supported OS.

HIP

What HIP is (not emulation) and how it maps to CUDA.
One source compiles for AMD or NVIDIA; hipify ports CUDA to HIP.

ROCm TOOLS

amd-smi, rocminfo, hipcc, rocprofv3 – the daily workflow
Commands cover monitor, query, compile, profile.

AMD
together we advance_

Public

GPU Introduction

What a GPU is, why it's the right tool for AI and HPC, and how the design tradeoff drives the economics.

- **What is a GPU? — parallel architecture at 50,000 ft**
- **CPU vs GPU — the fundamental latency/throughput tradeoff**
- **Why GPUs for AI and HPC — the economic argument**

AMD
together we advance_

Public

What Is a GPU?

GPU = Graphics Processing Unit – Originally Built to Shade Millions of Pixels in Parallel.

Born from graphics.
Each pixel's color computed independently ⟶ a naturally parallel workload.

~2004	GPGPU – researchers adapt the parallel engine for general compute.
~2012	Deep learning exploits GPUs for 10–100× faster training.
Today	Essentially all AI training and most HPC runs on GPUs.

KEY INSIGHT

GPUs aren't fast just because of parallelism. They hide memory latency by keeping **thousands of threads in flight** simultaneously.

THE RESULT

10–100× more compute per dollar and per watt than CPUs for **data-parallel workloads** - the economic driver behind every cloud GPU instance.

This course focuses on data-center GPUs (AMD Instinct™) – purpose-built for AI and HPC.

AMD
together we advance_

Public

CPU vs GPU – The Fundamental Tradeoff

Same Silicon Area, Two Different Design Philosophies. One Optimizes Latency, the Other Throughput.

CPU – LATENCY OPTIMIZED

- **Few powerful cores**
  8 – 128 cores, large caches (MBs per core).
- **Branch prediction, out-of-order execution**
  Built to chase one thread as fast as possible.
- **Good at**
  serial, branchy, irregular code.
- **Example**
  EPYC 9754 ~18-25 TFLOPS BF16

GPU – THROUGHPUT OPTIMIZED

- **Thousands of simple ALUs**
  Small caches, massive memory bandwidth.
- **No branch prediction, in-order**
  Designed for parallel throughput, not latency.
- **Good at**
  data-parallel SIMT workloads – one instruction, many threads.
- **Example**
  MI300X ~1300 TFLOPS FP16/BF16

CPU BF16 estimate: 128 cores * frequency * ~64 BF16 FLOP/cycle/core (base: 2.25 GHz, boost: 3.1 GHz)

AMD
together we advance_

Public

Why GPUs for AI and HPC?

Both workloads expose massive parallel numerical work.

AI TRAINING & INFERENCE

Neural networks use matrix, convolution, attention, reduction, and element-wise operations.

Many calculations run independently, or with limited coordination, across large datasets.

Training repeats this work at scale while Inference applies trained model to many inputs or requests.

HIGH-PERFORMANCE COMPUTING

Weather simulation, molecular dynamics, CFD, genomics.

Huge grids or particle systems where many updates can be computed independently.

Simulations apply similar numerical operations across many points, particles, and time steps.

Common thread: GPUs convert massive parallelism into practical performance.

AMD
together we advance_

Public

The AMD ROCm™ Software Stack

A layered look at what runs between your Python app and the GPU silicon.

- **What is ROCm™? – AMD's GPU programming software stack**
- **AMD ROCm™ ecosystem – from apps down to hardware**
- **AMD ROCm™ – libraries · tools · runtime**
- **Releases & compatibility – AMD ROCm™ 7.x, LLVM targets, supported GPUs & OSes**

AMD
together we advance_

Public

What Is AMD ROCm™?

AMD's Open Software Stack for GPU Computing.

AMD ROCm™ Software Stack
runtime, compilers, libraries, tools

+

AMD GPU
Instinct / Radeon

=

Accelerated Compute
AI, HPC, GPU applications

OPEN SOFTWARE STACK

- **Core ROCm components are published openly.**

  Inspect, build, debug, and contribute.

CUDA SOURCE PORTABILITY

- **HIP provides a CUDA-like C++ programming model.**

  hipify helps migrate existing CUDA source code to HIP.

DEPLOYED AT SCALE

- **Used across hyperscale AI, public cloud, and exascale HPC.**

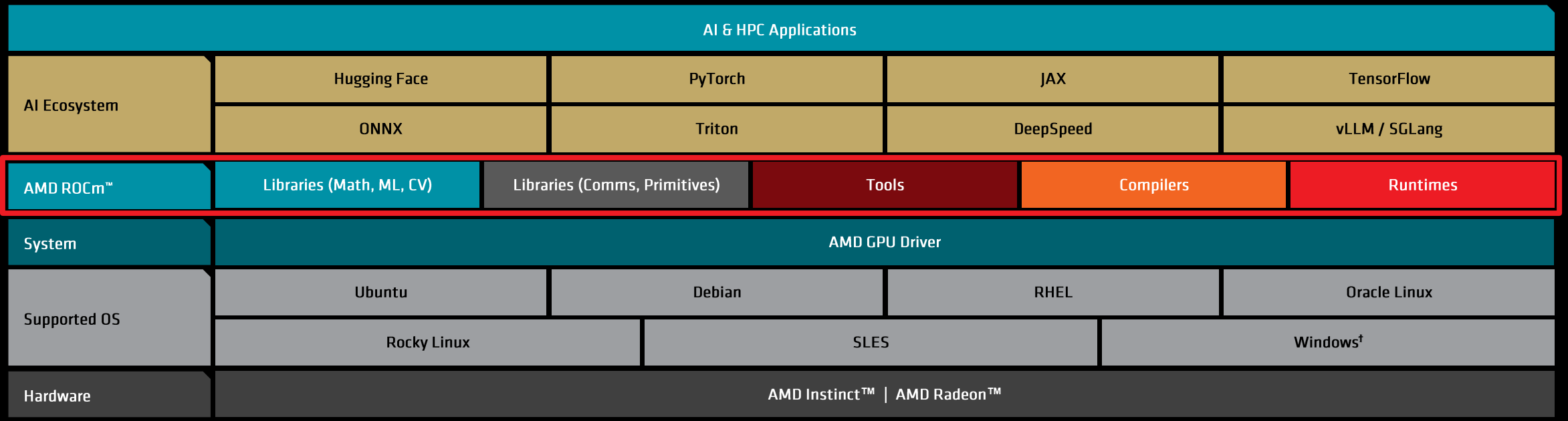
  Examples: Azure MI300X, OCI MI355X, El Capitan (MI300A) and Frontier (MI250X) Supercomputers

AMD
together we advance_

Public

AMD ROCm™ Ecosystem

A layered stack from applications to AMD GPUs



† Limited support.
Sources: AMD ROCm 7.2.4 Documentation, HIP SDK for Windows

AMD
together we advance_

Public

AMD ROCm™

Core ROCm components, grouped by role.

LIBRARIES	ML / CV	CK · MIGraphX · MIOpen · MIVisionX · RPP · rocAL · rocDecode · rocJPEG · rocPyDecode	*Deep learning, inference, vision, media*
	COMM	RCCL · rocSHMEM	*Multi-GPU / multi-node collective communication*
	MATH	hipBLAS · hipBLASLt · hipFFT · hipRAND · hipSOLVER · hipSPARSE · rocBLAS · rocFFT · rocSOLVER · rocWMMA · Tensile · hipfort · half	*Linear algebra, FFT, RNG, solvers, sparse – the compute workhorse*
	PRIMITIVES	hipCUB · hipTensor · rocPRIM · rocThrust	*Sort, scan, reduce, parallel algorithms*
TOOLS		AMD SMI · ROCm™ SMI · rocminfo · rocprofv3 · Compute Profiler · Systems Profiler · HIPIFY · ROCgdb	*Monitor · query · profile · debug · port CUDA code*
COMPILERS		HIPCC · ROCm™ compilers · FLANG	*GPU compiler drivers and offloading*
RUNTIME		HIP Runtime	*Portable GPU programming API*

source: https://rocm.docs.amd.com/en/docs-7.2.4/what-is-rocm.html

AMD
together we advance_

Public

AMD ROCm™ Releases & Compatibility

Which AMD GPUs map to which architecture and LLVM target.

Product		Architecture	LLVM Target
Instinct	MI355X · MI350X · MI350P	CDNA 4	gfx950
	MI325X · MI300X · MI300A	CDNA 3	gfx942
	MI250X · MI250 · MI210	CDNA 2	gfx90a
	MI100	CDNA 1	gfx908
Radeon	RX 9070/9060 · AI PRO R9700	RDNA 4	gfx1200/1201
	RX 7900/7800/7700 · PRO W7900/W7800	RDNA 3	gfx1100/1101

For a given GPU family, hardware target is fixed; ROCm support is release-specific
For MI300X, target gfx942 at compile time or use `--offload-arch=gfx942` for lower-level build flows

source: https://rocm.docs.amd.com/en/docs-7.2.4/compatibility/compatibility-matrix.html

Public

HIP – Your Programming Model

What HIP is and how one source compiles for AMD or NVIDIA.

HIP – Heterogeneous-compute Interface for Portability

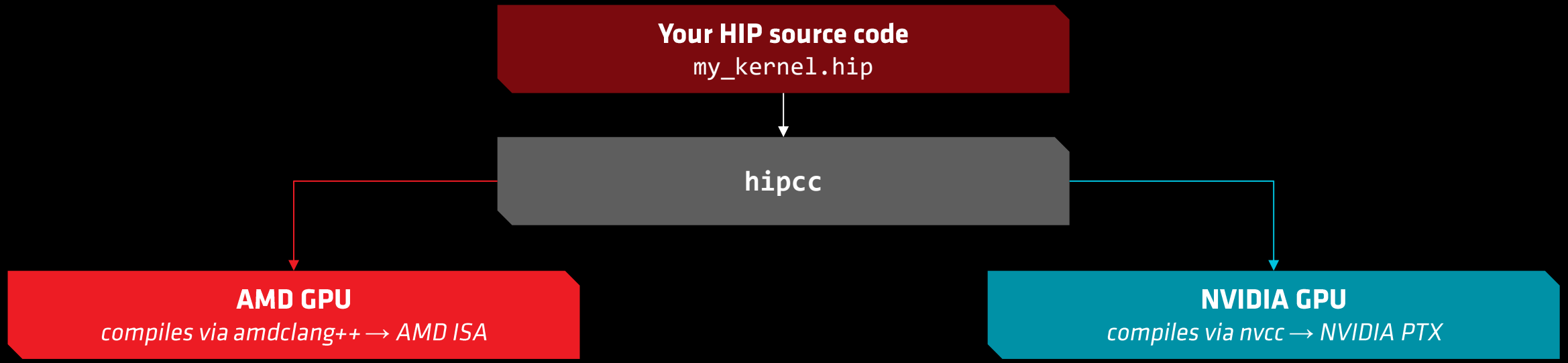
HIP vs CUDA – side by side

AMD
together we advance_

Public

HIP – Heterogeneous-compute Interface for Portability

Maintain a single source code base that can be compiled for AMD or NVIDIA GPUs.



SAME SOURCE

Same .hip file compiles on either platform – no #ifdefs, no per-vendor branches.

CUDA-LIKE SYNTAX

__global__, <<<grid,block>>>, threadIdx, blockIdx – all the familiar CUDA primitives.

CUDA PORTING PATH

Run HIPIFY tools to translate standard CUDA API to HIP

AMD
together we advance_

Public

HIP vs CUDA — Side by Side

Code fragment of Vector-add Kernel in HIP and CUDA. Only the API Prefix Differs.

HIP (AMD)

```
#include <hip/hip_runtime.h>
__global__ void add(float* a, float* b, float* c) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i<N) c[i] = a[i] + b[i];
}
int main() {
  float *a, *b, *c;
  hipMalloc(&a, N*sizeof(float));
  hipMalloc(&b, N*sizeof(float));
  hipMalloc(&c, N*sizeof(float));
  hipMemcpy(a, h_a, N*sizeof(float), hipMemcpyHostToDevice);
  hipMemcpy(b, h_b, N*sizeof(float), hipMemcpyHostToDevice);
  add<<<blocks, threads>>>(a,b,c);
  hipMemcpy(h_c, c, N*sizeof(float), hipMemcpyDeviceToHost);
}
```

CUDA (NVIDIA)

```
#include <cuda_runtime.h>
__global__ void add(float* a, float* b, float* c) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if(i<N) c[i] = a[i] + b[i];
}
int main() {
  float *a, *b, *c;
  cudaMalloc(&a, N*sizeof(float));
  cudaMalloc(&b, N*sizeof(float));
  cudaMalloc(&c, N*sizeof(float));
  cudaMemcpy(a, h_a, N*sizeof(float), cudaMemcpyHostToDevice);
  cudaMemcpy(b, h_b, N*sizeof(float), cudaMemcpyHostToDevice);
  add<<<blocks, threads>>>(a,b,c);
  cudaMemcpy(h_c, c, N*sizeof(float), cudaMemcpyDeviceToHost);
}
```

Kernel body is IDENTICAL – only the API prefix differs.
hipify-perl (ships with AMD ROCm™) renames CUDA APIs automatically.

AMD
together we advance_

Public

Tools

4 command-line tools cover the full AMD ROCm ™ development loop – from 'is the GPU alive?' to 'why is this kernel slow?'

- **amd-smi – monitor**
- **rocminfo – query**
- **hipcc – compile HIP code**
- **rocprofv3 – profile kernels and counters**

AMD
together we advance_

Public

AMD ROCm™ Tool Tour

Four Tools Cover The Full Cycle: Monitor → Query → Compile → Profile.

TOOL	WHAT IT DOES	WHEN YOU USE IT
`amd-smi`	GPU dashboard: temp, clocks, memory, power, utilization.	*When you want a quick health check – is it busy? overheating?*
`rocminfo`	Lists GPUs and CPUs: name, CU count, ISA, wavefront, clocks.	*First thing after install – does ROCm see the GPU?*
`hipcc`	HIP compiler – auto-detects GPU, emits native code.	*Every compile: hipcc -O3 -o out in.hip*
`rocprofv3`	Kernel times, HW counters, occupancy metrics.	*Profiling: rocprofv3 --stats --kernel-trace -- ./app*

These four cover: monitor → query → compile → profile

AMD
together we advance_

Public

Module 1 Summary

GPU INTRODUCTION

GPUs are massively parallel, throughput-optimized hardware. On data-parallel workloads they can deliver order-of-magnitude speedups over CPUs – and far more compute per dollar and per watt.

AMD ROCm™ STACK

AMD's open-source software stack that consists of a collection of drivers, development tools, and APIs used for programming and running on AMD GPUs.

HIP

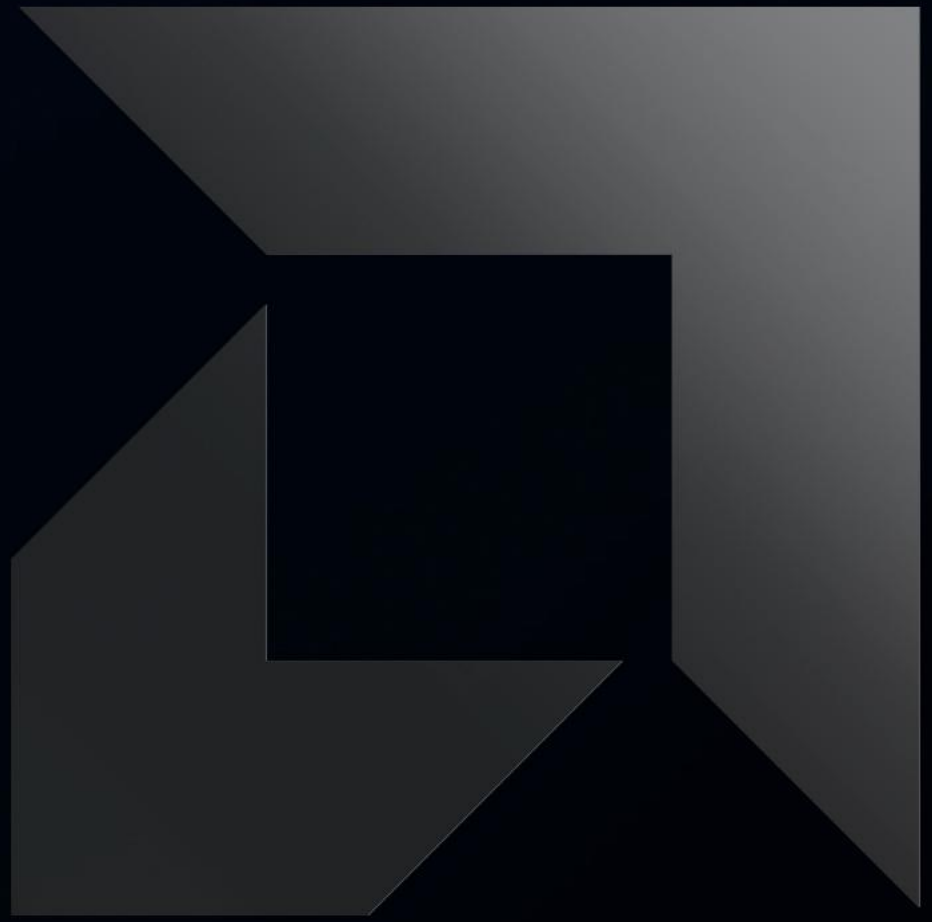
AMD's C++ runtime API and kernel language that allows developers to create portable source code that can be compiled for AMD or NVIDIA GPUs.

ROCm TOOLS

Tools included in the ROCm stack for monitoring and querying GPU, compiling HIP code, and understanding performance.

→ NEXT: Module 2 – AMD Instinct Architecture

AMD
together we advance_



ROCm™ Certification – Level 1

An Introduction to AMD GPUs and ROCm for AI and HPC

Module 2: AMD Instinct Architecture

AMD
together we advance_

Public

Module 2 Objectives

AMD INSTINCT

The Instinct lineage, the MI300 package down to one Compute Unit, and the on-chip memory hierarchy from registers to HBM.

GPU Programming and Execution Model

How your code maps to the chip: grid → block → wavefront → SIMD. The SIMT model, and how wavefronts hide memory latency.

CDNA vs RDNA

Two AMD GPU architectures — one for data-center compute, one for professional and consumer compute and graphics. Same HIP source compiles for both.

AMD
together we advance_

Public

The GPU – Family, Package, CU

From the Instinct lineage down to the engines and memory inside a single Compute Unit.

- AMD Instinct™ lineage
- MI300 anatomy – package to Compute Unit
- The SIMD unit – the CU's vector engine
- The MFMA matrix engine
- Memory hierarchy – registers to HBM

AMD
together we advance_

Public

AMD Instinct™ Lineage

Each new CDNA generation roughly doubles peak AI throughput, expands memory and bandwidth, and evolves precision support for modern workloads.

MI25/MI50	MI100	MI200	MI300/325	MI350/355	MI400
2017–18	2020	2021	2023–24	2025	H2 2026
GCN	*CDNA 1*	*CDNA 2*	*CDNA 3*	*CDNA 4*	*CDNA Next*
vector compute begins	first matrix cores	multi-die, strong FP64	3D chiplets, FP8	FP6/FP4	HBM4, UALink, 2nm

Key specs and peak performance by generation

MI300X	MI325X	MI350/355X	MI400 family	MI500
192 GB HBM3	256 GB HBM3E	288 GB HBM3E	432 GB HBM4	
5.3 TB/s	6.0 TB/s	8.0 TB/s	19.6 TB/s	
~5.2 PFLOPS FP8	~5.2 PFLOPS FP8	~10 PFLOPS FP8	~20 PFLOPS FP8	2027
		~20 PFLOPS FP4	~40 PFLOPS FP4	

PFLOPS listed as peak values with sparsity.

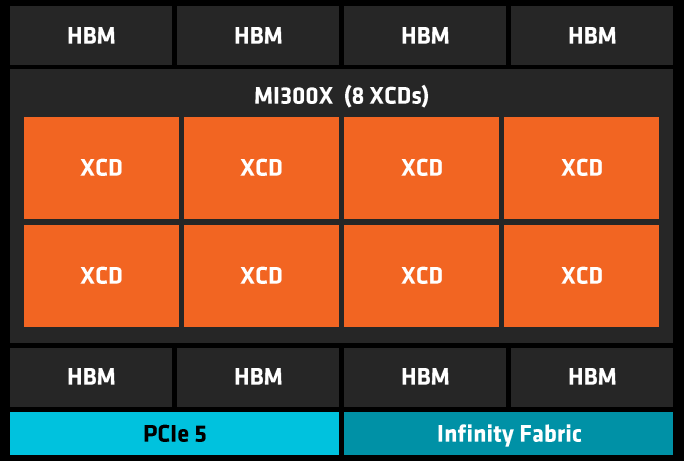
AMD
together we advance_

Public

MI300X – Package, XCD, Compute Unit

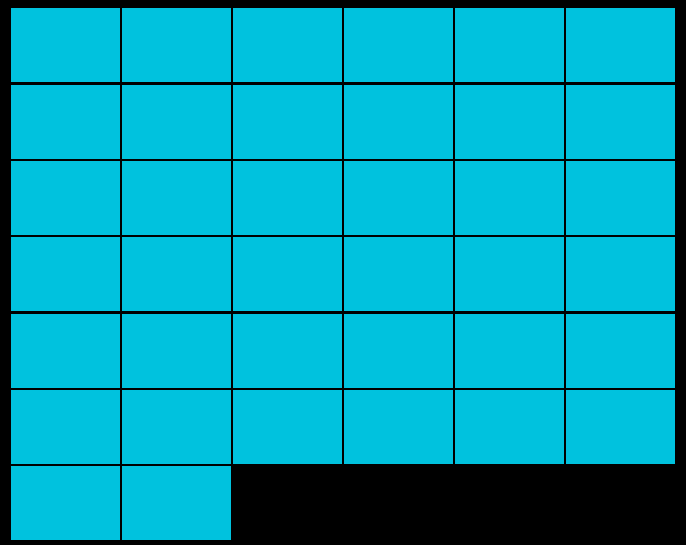
From the full board (package + HBM + I/O) down to one CU's internals.
Each panel zooms into the one before it.

1 FULL PACKAGE



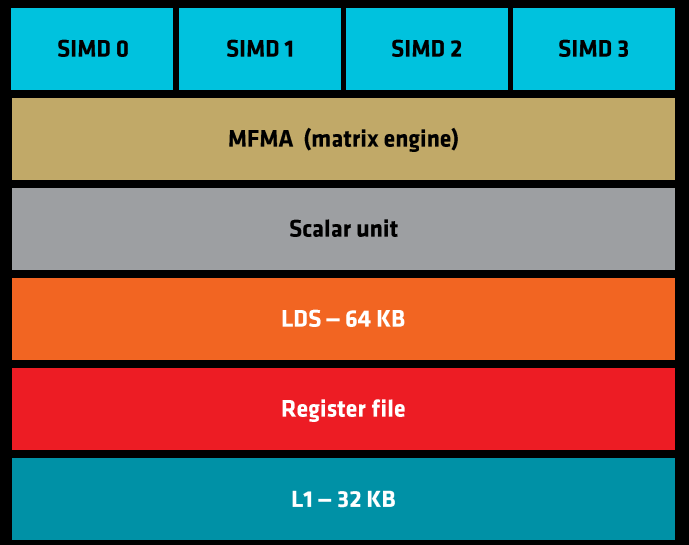
192 GB HBM3 · 8 stacks
256 MB Infinity Cache – on the I/O dies

2 ONE XCD



38 active CUs per XCD (304 active CUs total)
4 MB L2 cache – shared by the die's CUs

3 ONE CU



64 ALUs + MFMA
the building block

Package = 8 XCDs + 8 HBM3 stacks + PCIe + Infinity Fabric + Infinity Cache

Accelerator Complex Die (XCD) = 38 CUs + L2

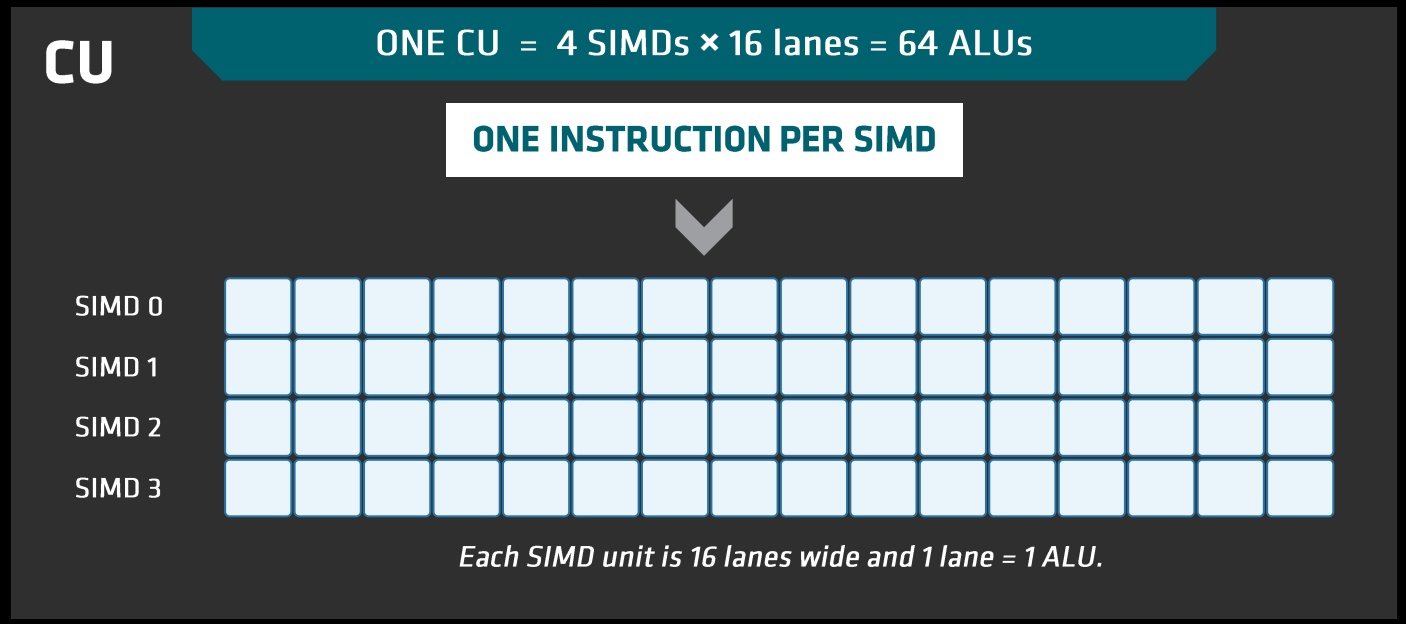
Compute Unit (CU) = 4 SIMDs × 16 ALUs + MFMA + LDS + L1

AMD
together we advance_

Public

SIMD Unit – The CU's Vector Engine

Each SIMD unit is 16 lanes wide – one lane is one ALU. A Compute Unit contains four of them.



16 LANES = 16 ALUs

Each lane is one ALU – the arithmetic logic unit that does the math. The 16 lanes run the same instruction on 16 different data elements.

FOUR SIMD UNITS PER CU

A Compute Unit packs 4 SIMD units – 64 ALUs in total.

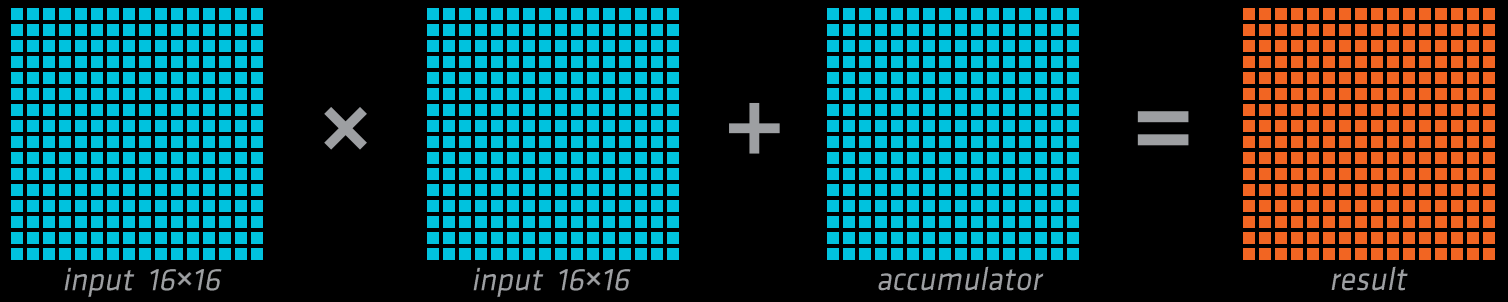
AMD
together we advance_

Public

MFMA – One Instruction, One Full Matrix Tile

The matrix engine computes a full multiply-accumulate tile (D = A × B + C) in a single instruction – the building block of every GEMM.

WHAT MFMA DOES



V_MFMA_F32_16x16x16_F16

FP16 inputs · FP32 accumulate

One instruction replaces thousands of scalar FMAs – about 4,000 for this 16×16 tile.

MFMA comes in several tile shapes (e.g., 16×16, 32×32); 16×16 shown here.

WHAT IT IS

Fused matrix op – one instruction does a full tile MMA

HOW IT WORKS

Runs on the Matrix Cores – 4 per CU, one per SIMD, sharing the vector registers.

WHY IT MATTERS

70–90% of AI compute is GEMM – rocBLAS / MIOpen use MFMA automatically

AMD
together we advance_

Public

Memory Hierarchy

Bandwidth climbs and latency drops the closer data sits to the CU.

FAST

REGISTERS (VGPRs)
Up to 256 VGPRs/thread
~100s of TB/s | ~0 cycles

LDS (shared memory)
64 KB per CU | ~10s of TB/s | ~tens of cycles

L1 CACHE
32 KB per CU | ~10s of TB/s | ~tens of cycles

L2 CACHE
4 MB per XCD (32 MB total) | ~34 TB/s | ~hundreds of cycles

INFINITY CACHE
256 MB | ~17 TB/s | ~hundreds of cycles (> L2)

SLOW

HBM (global memory)
192 GB | ~5.3 TB/s | ~hundreds of cycles (slowest)

Why it matters

BANDWIDTH and LATENCY GAP
On-chip memory delivers far more bandwidth and far lower latency than HBM – roughly 10× or more, registers to HBM. If a thread's data isn't on-chip it must reach out to global HBM – which is far slower.

THE RULE OF THUMB
VGPRs > LDS > L1 > L2 > Infinity Cache > HBM – keep hot data as close to the CU as possible.

REUSE > RE-FETCH
If a kernel reuses data, hold values in registers through tiling, stage shared data in LDS, or let the caches catch the rest, rather than re-fetching from HBM.

Capacities are MI300X. L2 / Infinity Cache / HBM bandwidths are AMD specs; register / LDS / L1 bandwidths and latencies are rough estimates, not measured.

AMD
together we advance_

Public

GPU Programming and Execution Model

How the GPU actually runs your code – from the kernels you write down to how wavefronts hide memory latency.

- **GPU Kernels and SPMD**
- **Thread Hierarchy and Mapping Threads to the Chip**
- **SIMT – The GPU Execution Model**
- **Wavefronts and Latency Hiding**

AMD
together we advance_

Public

GPU Kernels and SPMD

Thousands of threads execute the kernel on different data.

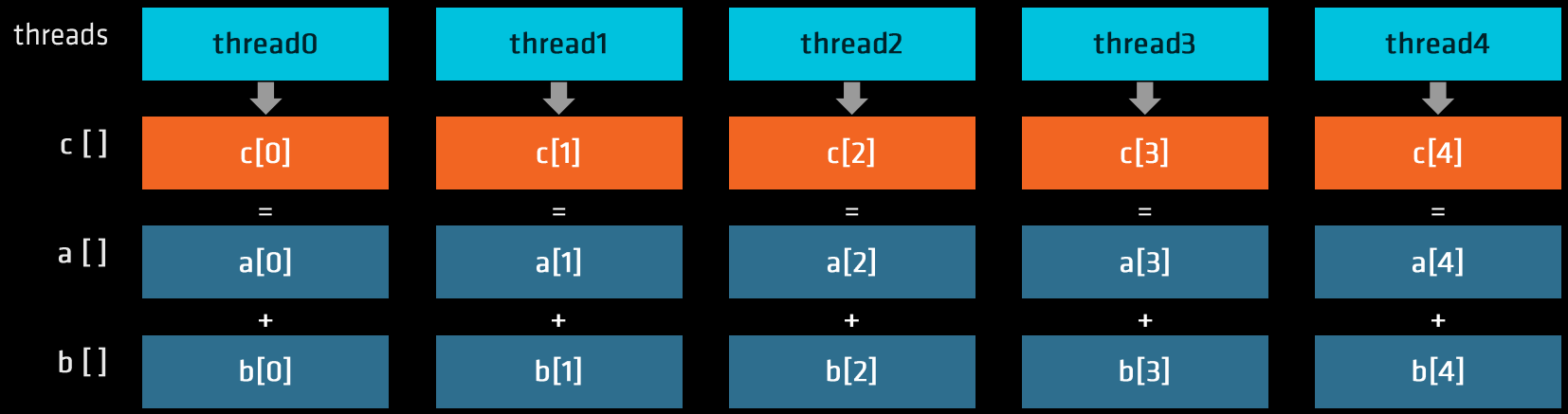
KERNEL
A function written to execute on the GPU. When the host program launches the kernel, it creates thousands of threads that process different data in parallel.

SPMD
Single Program, Multiple Data – the same kernel runs across many threads at once, each on its own data. It's the model you write in.

Example: vector addition – the element-wise sum of two arrays.

On the GPU, each thread executes the addition on a different data element of the arrays: thread `i` → `c[i] = a[i] + b[i]`

The thread's index picks the data: `i = blockIdx.x * blockDim.x + threadIdx.x`

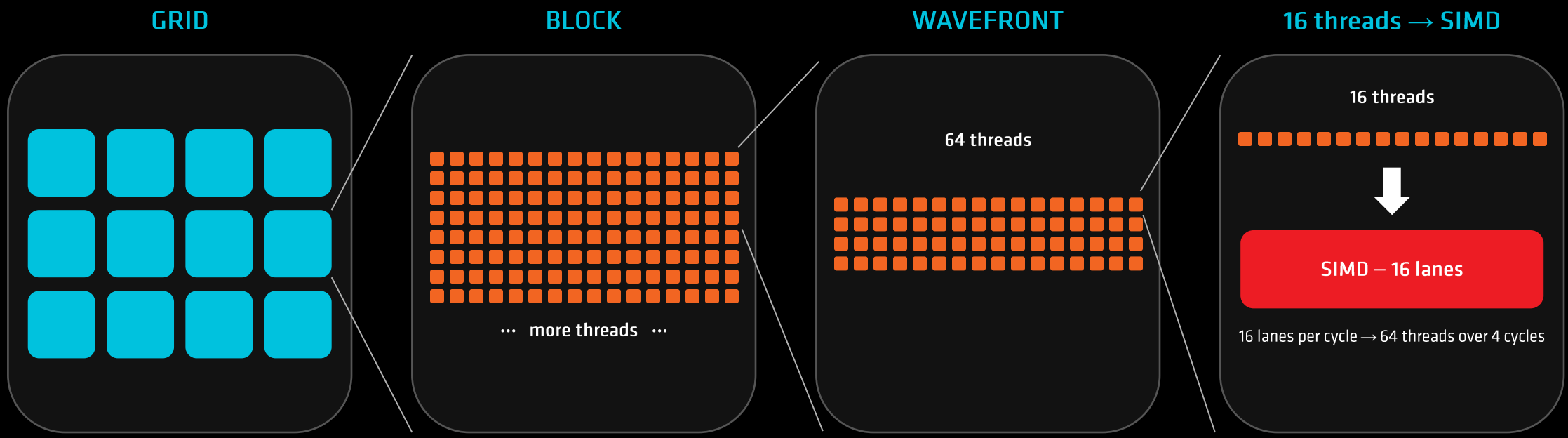


AMD
together we advance_

Public

Thread Hierarchy and Mapping Threads to the Chip

How the GPU actually runs your code.



GRID – array of blocks

BLOCK – up to 1024 threads

WAVEFRONT – 64 threads executed in lockstep on the hardware

SIMD – 16 lanes; 64 ⟶ 4 cycles

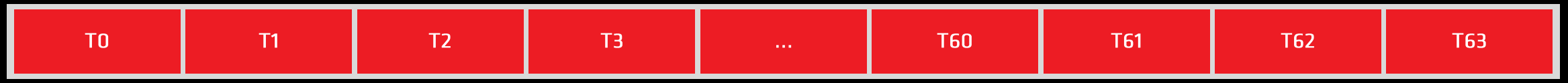
AMD
together we advance_

Public

SIMT – The GPU Execution Model

Single Instruction, Multiple Threads – one instruction carried out in lockstep by 64-thread wavefronts

1 WAVEFRONT = 64 threads in lockstep



SAME INSTRUCTION, OWN DATA

One instruction is issued to the wavefront; all 64 threads execute it at once – each on its own data, in its own registers.

SHARED PROGRAM COUNTER

The 64 threads share one program counter, so the wavefront advances in lockstep – one instruction at a time.

BRANCHES? HARDWARE MASKS LANES

When threads diverge on if/else, the hardware masks inactive lanes automatically.

Example: MI300X = 304 CUs × 64 ALUs – ≈20,000 threads executing simultaneously.

AMD
together we advance_

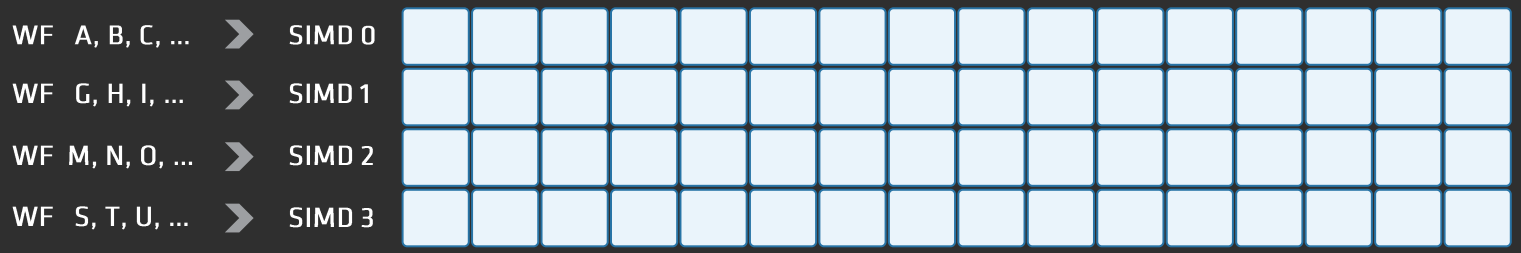
Public

Wavefronts & Latency Hiding

Keeping many wavefronts in flight hides memory latency.

RECALL: A wavefront is a group of 64 threads that executes the same instruction in lockstep on ONE SIMD

CU



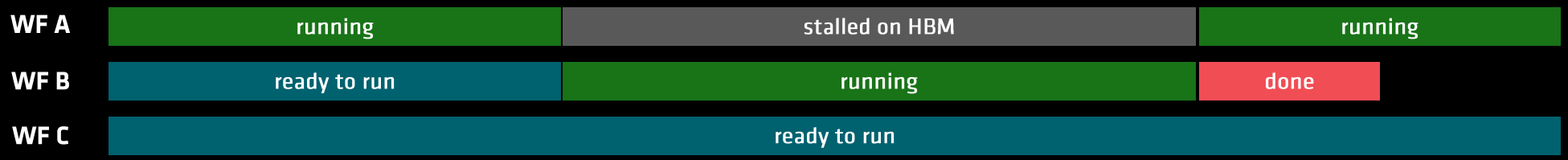
All 64 threads advance one instruction in lockstep – on ONE SIMD (16 lanes), 4 cycles per instruction.

ON MEMORY STALL
When wavefront A stalls on HBM, the scheduler switches to wavefront B in the next cycle – zero context-switch cost.

MORE WAVES = MORE HIDING
Each CDNA 3 CU can keep up to 32 wavefronts in flight – that's the depth of the latency-hiding pipeline.

WHY THOUSANDS OF THREADS
Not for parallelism alone – GPUs need many wavefronts ready so the CU is never idle while memory is in flight.

Latency hiding in action – three wavefronts running on a single SIMD over time



■ *green = running on a SIMD* ■ *blue = ready to run* ■ *grey = stalled on HBM* ■ *red = done*

Only one wavefront issues on a SIMD at a time; when it stalls on memory, the SIMD switches to a ready wavefront, so it's never idle.

AMD
together we advance_

Public

CDNA vs RDNA

AMD builds two GPU architectures – CDNA for the data center and RDNA for professional and consumer compute and graphics.

CDNA vs RDNA – Two Architectures

From Workstation to Data Center AI

AMD
together we advance_

Public

CDNA vs RDNA – Two Architectures

CDNA powers AMD Instinct in the data center; RDNA powers AMD Radeon for local compute and graphics.

	CDNA – AMD Instinct™ Data-center AI and HPC	RDNA – AMD Radeon™ Workstation AI and Graphics
Graphics hardware:	None	Rasterizers, ray tracing units, display outputs
Matrix acceleration:	MFMA (dedicated matrix cores)	WMMA (AI accelerators)
Memory:	HBM	GDDR
Wavefront:	64 threads	32 threads
gfx code examples:	gfx90a, gfx942, gfx950	gfx1100, gfx1201
GPU examples:	MI250X, MI300X, MI355X	Radeon RX 7900 XTX, Radeon AI Pro R9700

Same HIP source, different machine code: hipcc reads the target gfx code to pick opcodes.

AMD
together we advance_

Public

From Workstation to Data Center AI

Scaling AI compute from your desk to the data center, all on the same ROCm stack.

	Radeon AI Pro R9700 RDNA 4 · workstation AI	Instinct MI355X CDNA 4 · data-center AI & HPC
Memory	32 GB GDDR6	288 GB HBM3E
Bandwidth	640 GB/s	8 TB/s
FP16 (matrix)	191 TFLOPS	2.5 PFLOPS
Scales to	multiple cards over PCIe	8-GPU nodes · Infinity Fabric

Per GPU: ≈9× the memory · ≈12× the bandwidth · ≈13× the FP16 throughput — and Infinity Fabric lets a whole node act as one accelerator.

AMD
together we advance_

Public

Module 2 – Summary

AMD INSTINCT

MI100 → MI400: each generation adds compute, memory, and new precisions – now on an annual cadence.

A CU is 4 SIMD units (64 ALUs) + an MFMA engine; memory runs registers → caches → HBM.

GPU Programming and Execution Model

Your code maps grid → block → wavefront → SIMD; 64 threads run in lockstep (SIMT).

Many resident wavefronts hide memory latency – why GPUs want thousands of threads.

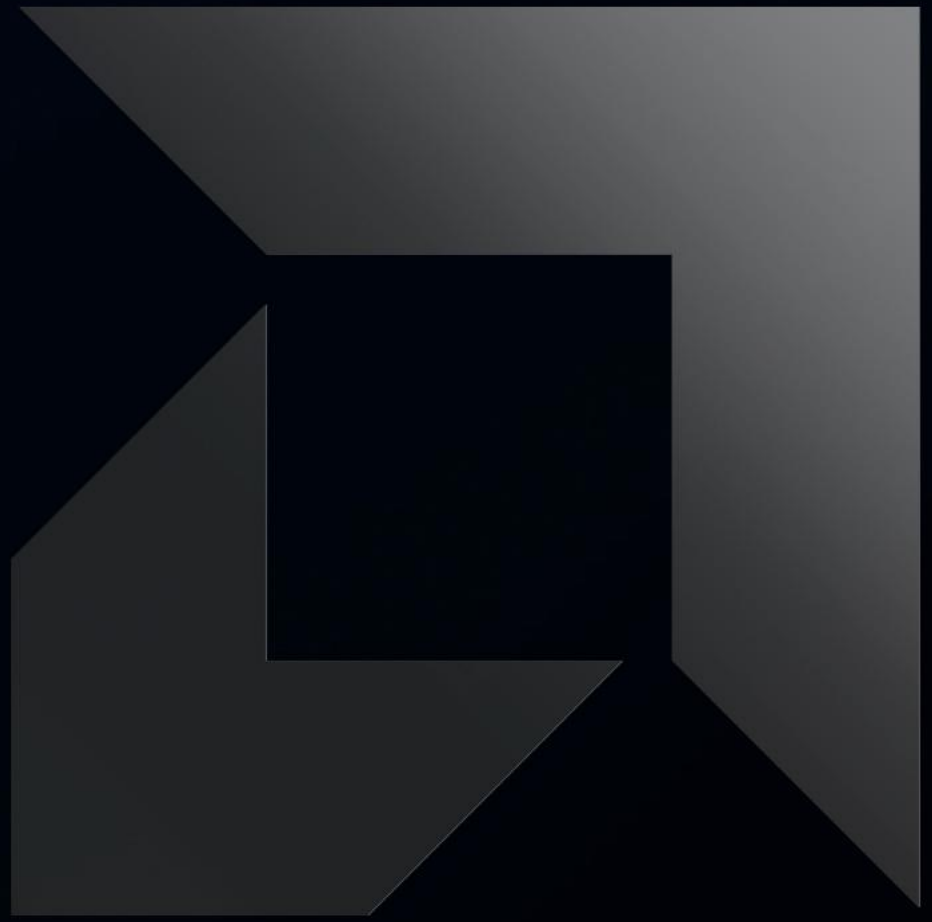
CDNA vs RDNA

CDNA is built for Instinct data center compute; RDNA powers Radeon for local AI and graphics.

Same HIP source, same ROCm stack – from a workstation card to a rack of data center GPUs.

→ NEXT: Module 3 – ROCm Libraries and AI Frameworks

AMD
together we advance_



ROCm™ Certification – Level 1

An Introduction to AMD GPUs and ROCm for AI and HPC

Module 3: ROCm Libraries and AI Frameworks

AMD
together we advance_

Public

Module 3 – Objectives

What you will learn in this module:



ROCM COMPUTE LIBRARIES

Pre-tuned building blocks for GEMM, convolutions, FFTs, solvers, RNG, sparse, and parallel primitives – plus Triton for custom kernels.



PYTORCH ON ROCM

The standard PyTorch workflow, native on AMD – the training loop and torch.compile().



SCALING & SERVING

Beyond a single GPU – multi-GPU and multi-node training with RCCL and inference with PyTorch and vLLM.

AMD
together we advance_

Public

AMD ROCm™ Compute Libraries

Optimized building blocks for AI and HPC workloads on AMD GPUs.

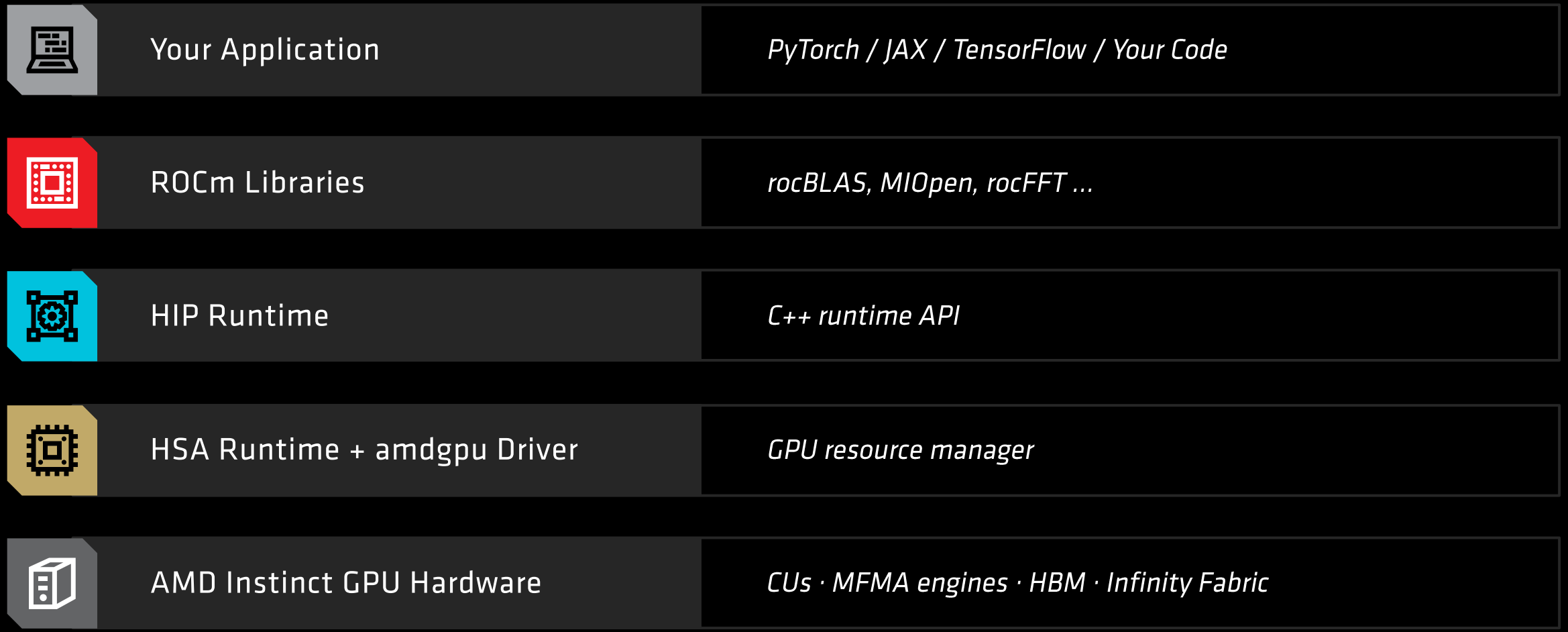
- Where libraries sit in the stack — and when to use them
- Compute libraries — rocBLAS · MIOpen · rocFFT · rocSOLVER · rocRAND · rocSPARSE · rocPRIM — plus Triton for custom kernels

AMD
together we advance_

Public

Where Libraries Sit in the Stack

Recall the ROCm stack from Module 1. Today's focus is the library layer:



AMD
together we advance_

Public

ROCm Compute Libraries – What They Do and Where They're Used

Map your workload to the right library.

LIBRARY	WHAT IT DOES	WHERE IT'S USED
rocBLAS / hipBLASLt	Provides dense linear algebra operations, such as GEMM	*Accounts for much of the computation in modern AI*
MIOpen	Provides deep learning primitives, such as convolution	*Convolution layer in CNNs*
rocFFT	Provides Fast Fourier Transforms in 1D, 2D, and 3D	*Signal processing, spectral methods, physics*
rocSOLVER	Provides matrix factorizations and solvers	*Scientific computing, PCA, regression*
rocRAND	Generates pseudo-random and quasi-random numbers	*Simulation, sampling, Monte Carlo*
rocSPARSE	Provides sparse matrix and vector operations	*Scientific solvers, graph workloads, sparse data needs*
rocPRIM / hipCUB	Provides parallel primitives, such as scan, sort	*Custom kernel building blocks*

AMD
together we advance_

Public

rocBLAS/hipBLASLt – Dense Linear Algebra

The BLAS standard, accelerated. One of the most-called libraries in deep learning.

WHAT IT DOES

Level 1	Vector–vector	*AXPY, DOT, SCAL, NRM2*
Level 2	Matrix–vector	*GEMV, TRSV*
Level 3	Matrix–matrix	*GEMM, TRSM*

GEMM: $D = \alpha * A * B + \beta * C$ (= A * B when α=1, β=0)

In every linear layer and attention head — and many convolutions.

WHY IT'S FAST

- Calls MFMA matrix engine
- Tile sizes are selected per CDNA generation to maximize data reuse in LDS and saturate the MFMA engines
- Software pipeline depth is tuned to hide memory latency behind compute and keep the MFMA units busy

hipBLASLt – GEMM-focused, with programmable layouts/types/heuristics, fused bias + activation, and FP8 on CDNA 3+.

$Y = \text{Activation}(\alpha \cdot A \cdot B + \beta \cdot C + \text{bias})$

AMD
together we advance_

Public

THEORY

MIOpen – Deep Learning Primitives

AMD's analog to cuDNN. Used by PyTorch under the hood for conv & batch-norm.

OPERATIONS

Convolution	*forward + backward, 2D / 3D, multi-algo*
Batch norm	*training and inference modes*
Pooling	*max / average, with optional indices*
Activations	*ReLU, GELU (via graph API)*
RNN / LSTM / GRU	*fused recurrent cells*
Softmax	*fast, accurate, and log-softmax*

Same role as cuDNN

KEY FEATURES

Multi-algorithm dispatch	*Example: Winograd · GEMM · direct · implicit-GEMM - picked per shape*
Two-tier autotune	*FindDb chooses algorithm · PerfDb tunes its parameters*
Multi-precision	*fp32 · fp16 · bf16 · int8*
Operator fusion	*conv + bias + activation in a single kernel*

WHY FUSION MATTERS

Without fusion	conv → *HBM* → bn → *HBM* → relu
With fusion	conv + bn + relu (reduces off-chip memory access, avoids kernel launch overhead)

AMD
together we advance_

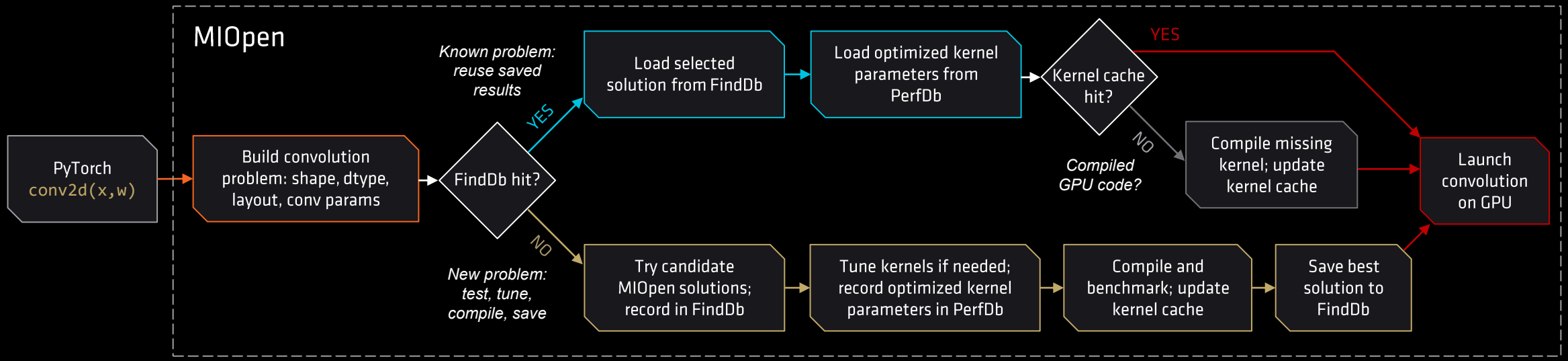
Public

MIOpen Autotune – Learn Once, Reuse Later

First time: test, tune and compile. Later: look up and launch.

FindDb	PerfDb	Kernel cache
Which solution should be used?	*How should it be tuned?*	*Is the GPU code already compiled?*
Remembers the best MIOpen implementation for a convolution problem.	Remembers low-level kernel tuning choices, such as tile size or block size.	Reuses compiled GPU kernels to avoid recompilation.

When PyTorch calls conv2d() on ROCm, the convolution is handed to MIOpen. Two cases: known problem and new problem.



AMD
together we advance_

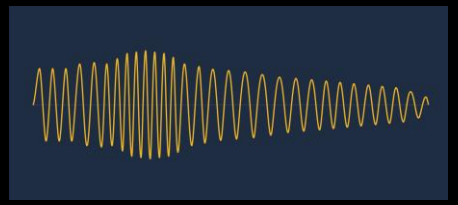
Public

rocFFT – Fast Fourier Transforms

Time domain ↔ frequency domain. Backbone of signal processing, physics, spectral solvers.

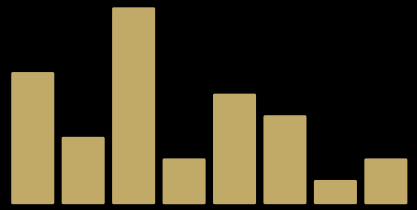
TIME → FREQUENCY

Time-domain signal



FFT

Frequency spectrum



WHAT rocFFT SUPPORTS

`1D / 2D / 3D`	*arbitrary lengths*
`Real & Complex FFTs`	*R2C / C2R / C2C*
`Batched FFTs`	*process N signals at once*
`Precisions`	*FP16 / FP32 / FP64*

WHERE YOU USE IT

Signal processing
audio, radar, telecom

Physics simulations
fluid dynamics, electromagnetics

Medical imaging
MRI reconstruction

AMD
together we advance_

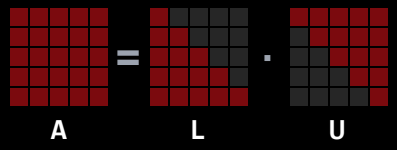
Public

rocSOLVER – Linear Algebra Decompositions

Each decomposition simplifies further calculations. Patterns show triangular, diagonal, and orthonormal structure.

LU

$A = L \cdot U$



Reveals:

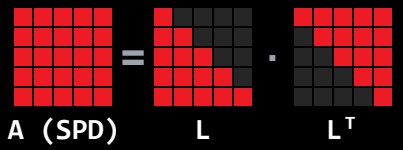
Lower- and upper-triangular factors.

Solves:

General linear systems $A \cdot x = b$

Cholesky

$A = L \cdot L^T$



Reveals:

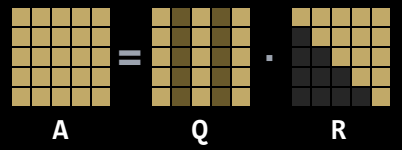
For SPD matrices: only one triangle needs storing. Half the work of LU.

Solves:

Symmetric pos-def systems · normal eqns · GPs

QR

$A = Q \cdot R$



Reveals:

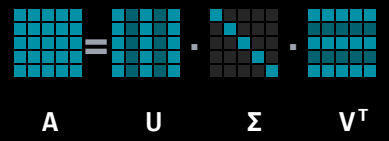
Q has orthonormal columns ($Q^T Q = I$). Stable solving without inverses.

Solves:

Least squares · regression · over-determined systems

SVD

$A = U \cdot \Sigma \cdot V^T$



Reveals:

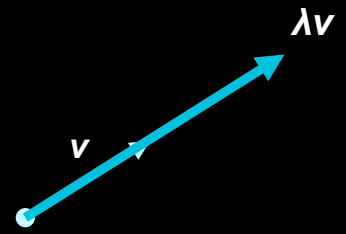
Singular values in Σ rank importance. Drop the small ones ~ compression.

Solves:

PCA · low-rank approx · recommenders · denoising

Eigenvalue

$A \cdot v = \lambda \cdot v$



Reveals:

Special directions v that A only stretches. λ = stretch factor.

Solves:

Spectral analysis · PageRank · stability · dynamics

VISUAL LEGEND – what each matrix pattern means

- **filled cell** — *non-zero entry*
- **lower triangular** — *L in LU, L in Cholesky*
- **upper triangular** — *U in LU, L^T in Cholesky, R in QR*
- **orthonormal cols** — *Q in QR, U in SVD*
- **orthonormal rows** — *V^T in SVD*
- **diagonal** — *Σ in SVD*

AMD
together we advance_

Public

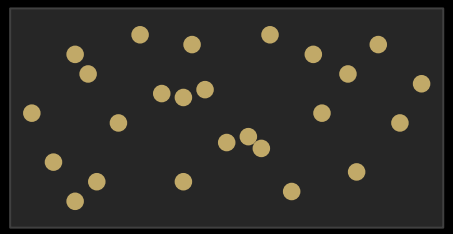
rocRAND – Pseudo vs Quasi-Random Generators

Two software RNG families with different goals. PRNG and QRNG are both deterministic software generators.

PRNG – PSEUDO-RANDOM

Goal: look random and statistically independent enough for simulation.

Deterministic algorithm + seed → reproducible sequence that passes statistical randomness tests.



clustered — has gaps

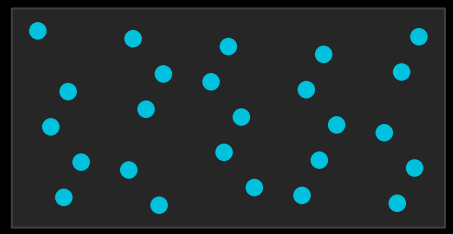
`XORWOW`	*xorshift; fast, simple bit ops*
`MT19937`	*Mersenne Twister*
`MTGP32`	*MT variant tuned for GPUs*
`Philox`	*counter-based, parallel-friendly*
`MRG32K3A / MRG31K3P`	*combined multiple recursive*
`LFSR113`	*linear-feedback shift register*
`ThreeFry`	*counter-based, Threefish cipher*

Best for: *simulation · sampling · randomized algorithms · stochastic ML kernels*

QRNG – QUASI-RANDOM

Goal: fill the space uniformly (low-discrepancy).

Points are constructed to reduce gaps and clumping. No seed like PRNG — indexed by dimension and offset.



uniform — no clumping

`Sobol32`	*32-bit Sobol low-discrepancy*
`Sobol64`	*64-bit; higher-resolution output*
`Scrambled Sobol32`	*scrambled for randomization; with low discrepancy*
`Scrambled Sobol64`	*scrambled 64-bit variant*

All four are based on the Sobol sequence (base-2 partitioning of the unit interval).

Best for: *Quasi-Monte Carlo · numerical integration · ray tracing · finance · option pricing*

QRNG = Quasi-Random Number Generators (not Quantum, as in hardware/crypto)

AMD
together we advance_

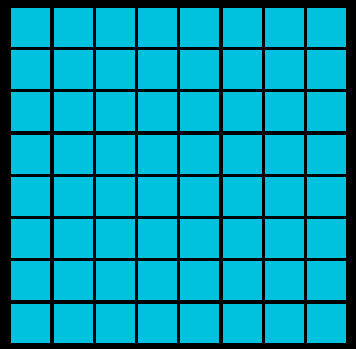
Public

rocSPARSE – Sparse Matrix Operations

When matrices are mostly zeros – often >99% in graphs, recommenders, and large scientific systems.

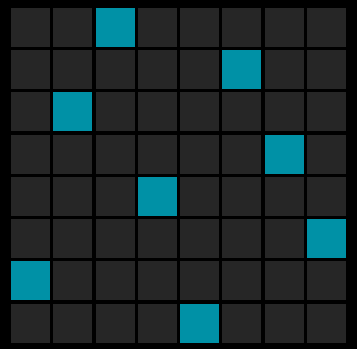
Core ops: SpMV · SpMM · SpGEMM · SDDMM · Sparse solves · Format conversion

DENSE vs SPARSE – WHY IT MATTERS



Dense

64/64 stored



Sparse

8/64 stored — in reality >99% zeros

MEMORY SAVED – concrete example

10,000 × 10,000 matrix, 0.1% density (typical GNN adjacency):

```
Dense (fp32):   400 MB
CSR sparse:     ~840 KB
```

→ ~475× smaller, and only the non-zeros are touched in compute.

STORAGE FORMATS

CSR	Compressed Sparse Row — most common · fast SpMV
CSC	Compressed Sparse Column — column-oriented operations
COO	Coordinate list — easy to assemble, sort, and convert
ELL	Fixed-width — best when row sizes are uniform
BSR	Block Sparse Row — for dense sub-blocks (e.g. FEM)
HYB	Hybrid ELL + COO — reg. rows in ELL, irreg. overflow in COO

Graph Neural Networks
adjacency matrices are sparse

Recommender systems
user × item matrices, mostly empty

Scientific solvers
PDEs, FEM, iterative methods

LLM attention
sparse/block-sparse research patterns

AMD
together we advance_

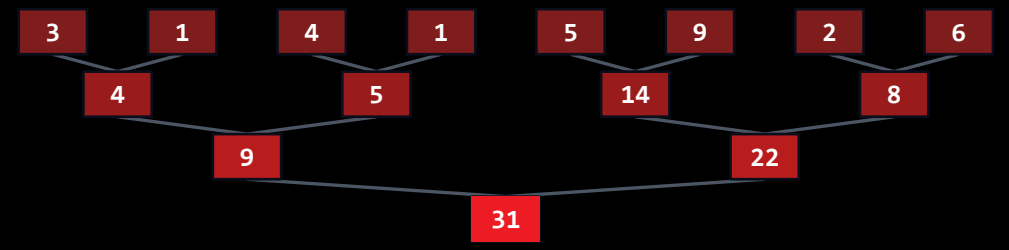
Public

rocPRIM / hipCUB – Parallel Primitives

Building blocks for whole-array operations and custom kernels. Replaces serial loops with parallel implementations on the GPU.

REDUCE

· *parallel tree, O(log N) depth*

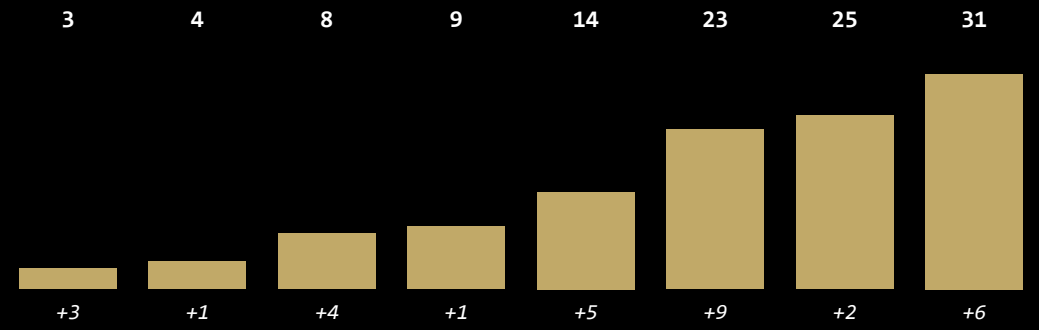


3 levels for 8 elements → $log_2 N = O(log N)$

Σ = 31

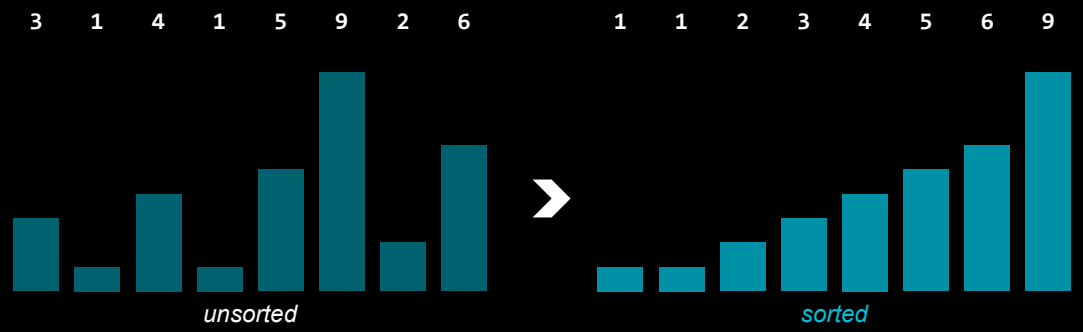
SCAN

(prefix sum) · running totals – each bar = sum so far



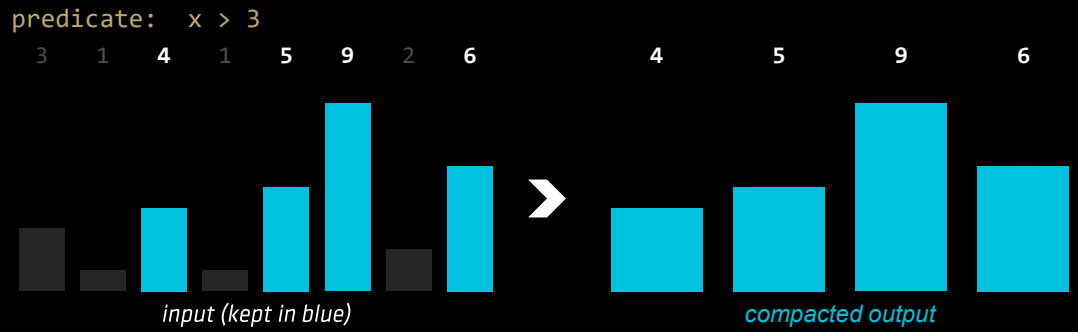
SORT

· *radix / merge – billions of keys/sec on Instinct*



SELECT

(filter) · keep elements where predicate is true → compacted output



Also: transform · histogram · merge · run-length encoding · partition.

AMD
together we advance_

Public

Triton Kernel Language and Compiler on AMD

Python-based kernel DSL for custom and fused GPU kernels.

BEYOND ROCm LIBRARIES: TRITON FOR CUSTOM KERNELS, HIP FOR FULL CONTROL

ROCm LIBRARIES

`rocBLAS · MIOpen · hipBLASLt · ...`

torch.matmul(a, b)

- ✓ easiest path for standard ops
- ⚠ *limited flexibility for custom fusion*

TRITON — custom kernels with less code

`Python DSL · @triton.jit`

fused, custom, portable kernels

- ✓ flexible path for custom kernels
- ⚠ *still requires GPU-kernel thinking*

RAW HIP

`HIP C++ kernels`

HIP C++ → amdclang/LLVM → AMD GPU binary

- ✓ maximum low-level control
- ⚠ *most code and tuning effort*

WHY USE TRITON

1 Python kernel DSL

Write block-level logic; Triton compiles it for the GPU

2 Portable across vendors

Same source can target AMD and NVIDIA backends; tuning may differ.

3 Auto-tuning built in

@triton.autotune benchmarks supplied configs and select best one for a shape.

4 Used by torch.compile

torch.compile uses TorchInductor; on AMD GPUs Triton is a key GPU codegen path.

AMD
together we advance_

Public

PyTorch on ROCm

The PyTorch you already know – the full ecosystem, the training loop, and torch.compile() – running natively on AMD ROCm.

- PyTorch on ROCm – Familiar Workflows and Ecosystem
- PyTorch Training Loop
- How torch.compile() Optimizes PyTorch

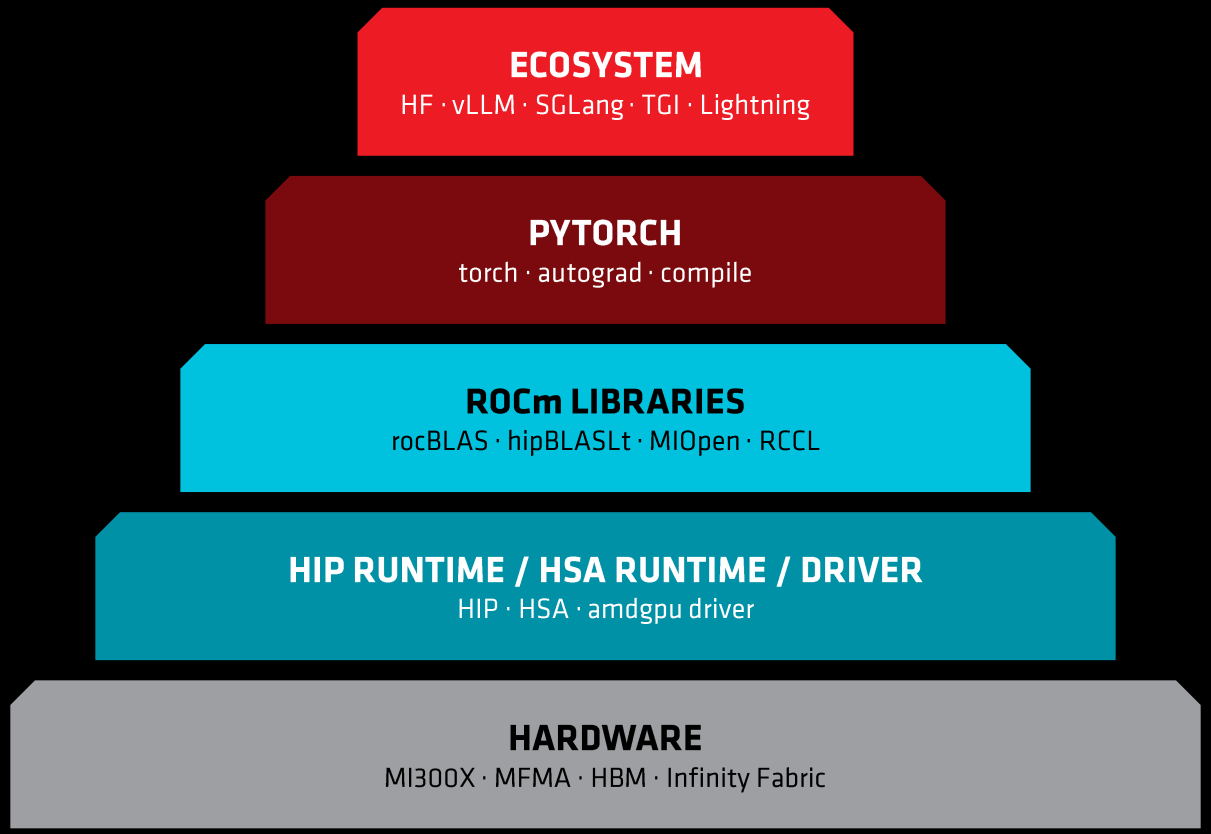
AMD
together we advance_

Public

PyTorch on ROCm – Familiar Workflows and Ecosystem

Same core PyTorch APIs and AI tools you already work with.

THE FULL STACK



COMMON AI WORKFLOWS

TRAIN & FINE-TUNE

- Single- & multi-GPU training on Instinct
- HF Transformers + PEFT
- LoRA / QLoRA for LLM fine-tuning
- FSDP / DeepSpeed ZeRO / Lightning

SERVE

- vLLM, SGLang, TGI – OpenAI-compatible APIs
- KV-cache management, continuous batching
- FP8 GEMM paths on CDNA 3 and newer
- Large throughput gains vs naive serving

RESEARCH

- Rapid prototyping with standard PyTorch workflows
- Lightning, Hydra, W&B, Diffusers, HF

AMD
together we advance_

Public

PyTorch Training Loop

Same common PyTorch training loop and torch.cuda API on ROCm.

STANDARD PYTORCH — same workflow on AMD

```
import torch
import torch.nn as nn

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
# ROCm uses the torch.cuda API and the 'cuda' device string

model     = resnet18().to(device)
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
loss_fn   = nn.CrossEntropyLoss()

for epoch in range(num_epochs):
    for inputs, labels in dataloader:

        inputs  = inputs.to(device)
        labels  = labels.to(device)

        outputs = model(inputs)
        loss    = loss_fn(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

HOW PYTORCH MAPS DOWN

`model(inputs)`	→ *MIOpen + rocBLAS/hipBLASLt GEMMs + HIP kernels*
`loss.backward()`	→ *Autograd → MIOpen + rocBLAS/hipBLASLt GEMMs + HIP kernels*
`optimizer.step()`	→ *HIP parameter-update kernels (SGD, AdamW, etc.)*
`inputs.to(device)`	→ *hipMemcpy / hipMemcpyAsync: host DRAM → GPU HBM*
`PyTorch op`	→ *GPU kernel → grid of thread blocks (workgroups) → wavefronts → execute on CUs*

Familiar PyTorch calls map down to ROCm libraries and AMD GPU execution.

AMD
together we advance_

Public

How torch.compile() Optimizes PyTorch

Captures PyTorch graph regions, fuses compatible operations, and generates optimized code.

WITHOUT torch.compile() — Eager Mode

`conv` `bias` `relu` `conv` `bias` `relu` `matmul`

Separate ops often mean separate GPU launches and temporary memory writes.

Eager: More launches and temporary data

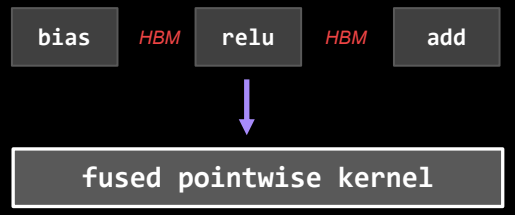
WITH torch.compile() — Graph Capture + Fused Eligible Ops

`conv` `fused bias + relu` `conv` `fused bias + relu` `matmul`

Captures graph regions, fuses eligible ops; fewer GPU launches and temporary memory writes.

Compiled: Fewer launches and temporary data

1 OPERATOR FUSION



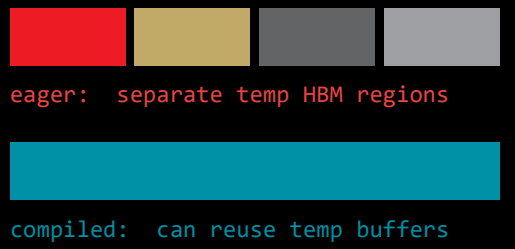
Compatible ops can be fused into one generated kernel, reducing intermediate HBM reads/writes.

2 FEWER LAUNCHES



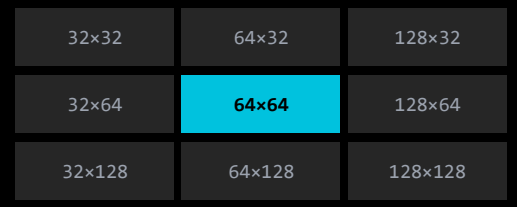
Each kernel launch has overhead. Fewer launches reduce CPU overhead and GPU idle time.

3 MEMORY PLANNING



Compiler can reuse temporary storage and avoid some temporary tensor writes.

4 OPTIONAL AUTO-TUNE



With mode="max-autotune", Inductor benchmarks candidate Triton configurations and selects the fastest for each shape.

```
ONE-LINE CHANGE  model = torch.compile(model)    # default backend = TorchInductor; on AMD it emits Triton kernels and calls ROCm libraries
```


AMD
together we advance_

Public

Scaling & Serving

Scaling across GPUs and nodes; running and serving models efficiently.

- RCCL – Multi-GPU and Multi-Node Communication
- PyTorch and vLLM for Inference on ROCm

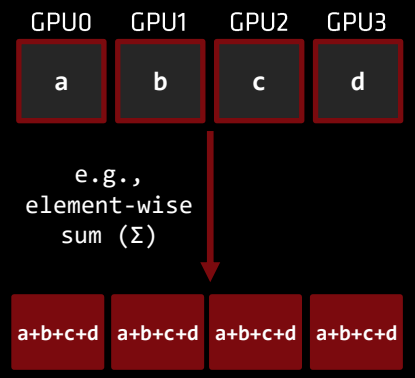
AMD
together we advance_

Public

RCCL – Multi-GPU and Multi-Node Communication

AMD's NCCL counterpart for ROCm GPU collectives in torch.distributed, DeepSpeed, and Megatron-LM.

All-Reduce

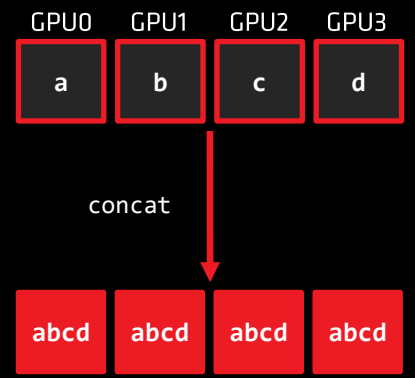


After (every GPU has Σ)

Each GPU contributes a same-size array; all receive the reduced array

Data-parallel training (gradient sync)

All-Gather

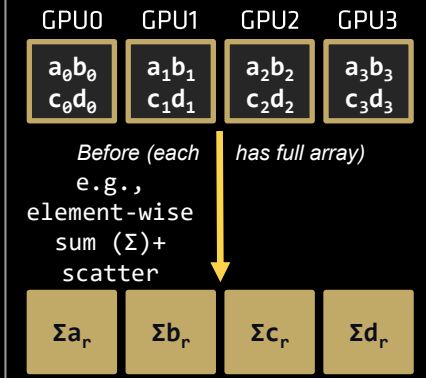


After (every GPU has the full array)

Each GPU contributes 1 chunk; all receive the concatenated array.

FSDP / ZeRO-3 (gather sharded parameters)

Reduce-Scatter

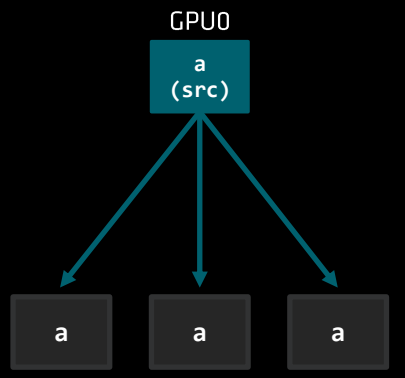


After (each holds 1 reduced chunk)

Reduce across GPUs; each receives 1 reduced chunk.

FSDP / ZeRO-3 (reduce and shard gradients)

Broadcast

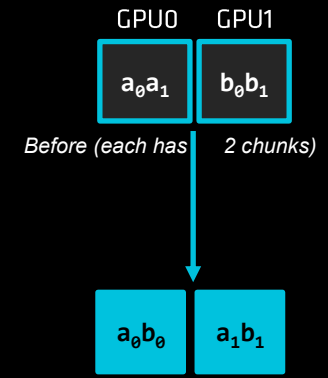


(all GPUs have a)

One GPU provides an array; all receive the same array.

Initial parameter / state sync

All-to-All



After (grouped by destination)

Each GPU contributes 1 chunk per destination and receives 1 from each destination

MoE token routing (tokens → expert GPU)

HARDWARE MAPPING – Topology-aware path and algorithm selection

Within a node

`xGMI / Infinity Fabric or PCIe`

Across nodes

`RoCE, InfiniBand, or TCP/IP`

Symbols denote arrays or array chunks; placing letters together denotes concatenation. Reduce-scatter subscripts identify source GPUs; all-to-all subscripts identify destination GPUs.

AMD
together we advance_

Public

PyTorch and vLLM for Inference on ROCm

PyTorch provides the model framework; vLLM uses it within a specialized LLM inference and serving engine.

DIRECT PYTORCH INFERENCE

General-purpose	*LLMs and other model types*
Flexible execution	*direct control over model and pipeline*
Custom workflows	*preprocessing, logic, and outputs*
Simple deployment	*scripts, applications, or services*

Well suited to general-purpose and custom inference workflows.

vLLM INFERENCE & SERVING

PagedAttention	*manages KV cache in page-like blocks*
Continuous batching	*dynamically batches active requests*
Quantization	*lower precision reduces memory use*
OpenAI-compatible API	*integrates with compatible clients*

Well suited to supported LLMs and high-throughput, concurrent serving.

RUN vLLM ON ROCm – official image, familiar CLI and API

```
# Pull the official ROCm vLLM image
docker pull vllm/vllm-openai-rocm:latest

# Start an OpenAI-compatible server
docker run --rm \
  --group-add=video \
  --cap-add=SYS_PTRACE \
  --security-opt seccomp=unconfined \
  --device=/dev/kfd \
  --device=/dev/dri \
  --ipc=host \
  -p 8000:8000 \
  -v /path/to/model:/model \
  vllm/vllm-openai-rocm:latest \
  --model /model
```


AMD
together we advance_

Public

Module 3 – Summary

Key takeaways from libraries, frameworks, multi-GPU and the compiler stack.



ROCM COMPUTE LIBRARIES

Use optimized ROCm Compute Libraries before writing a kernel; use Triton for custom kernels the libraries don't cover.



PYTORCH ON ROCM

Familiar calls map down to ROCm libraries and HIP; torch.compile() adds Triton fusion.

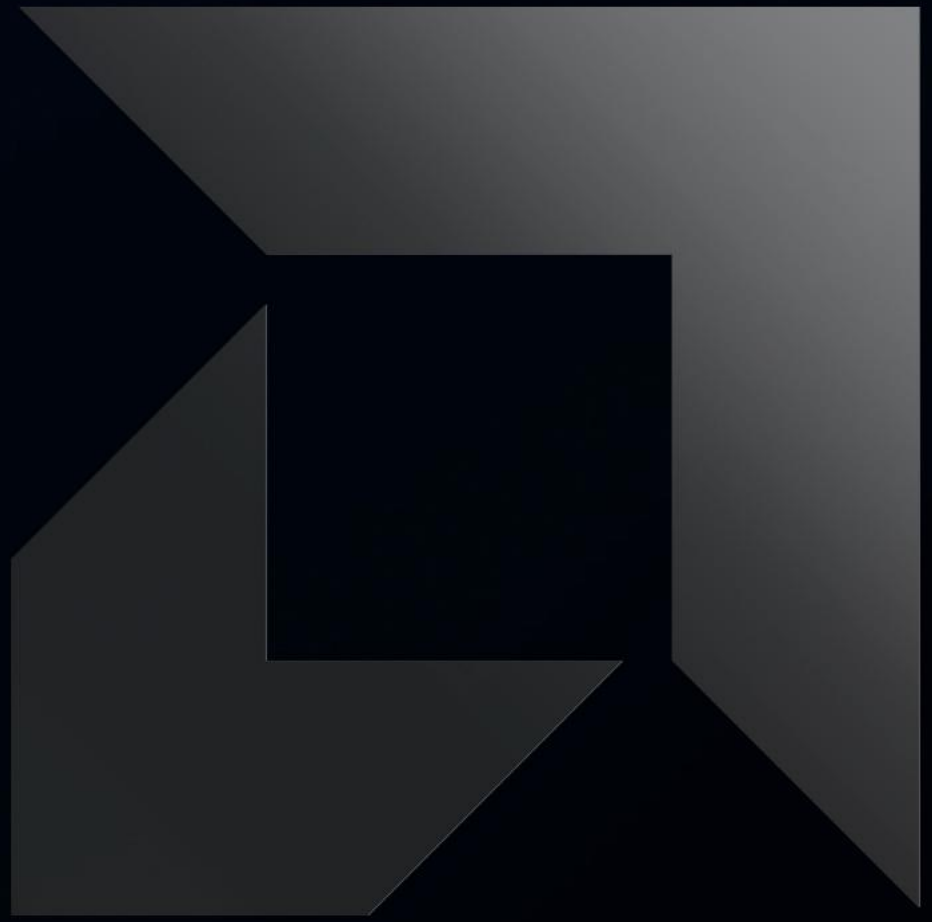


SCALING & SERVING

RCCL runs multi-GPU/multi-node collectives; for inference, PyTorch directly for flexibility, or vLLM for LLM serving at scale.

→ NEXT: Module 4 – HIP Programming and Porting from CUDA to HIP

AMD
together we advance_



ROCm™ Certification – Level 1

An Introduction to AMD GPUs and ROCm for AI and HPC

Module 4: HIP Programming and Porting from CUDA to HIP

AMD
together we advance_

Public

Module 4 – Objectives

What you will learn in this module:



HIP Programming Concepts

HIP concepts to get started.

Reminder of HIP structure and thread hierarchy; host-device programming.



HIP Programming

Introduction to HIP programming.

Vector addition example; error checking; timing HIP operations.



Porting from CUDA to HIP

Translating CUDA source to HIP

Automatic translation tools; vector addition porting example.

AMD
together we advance_

Public

HIP Programming Concepts

The HIP programming concepts needed to get started.

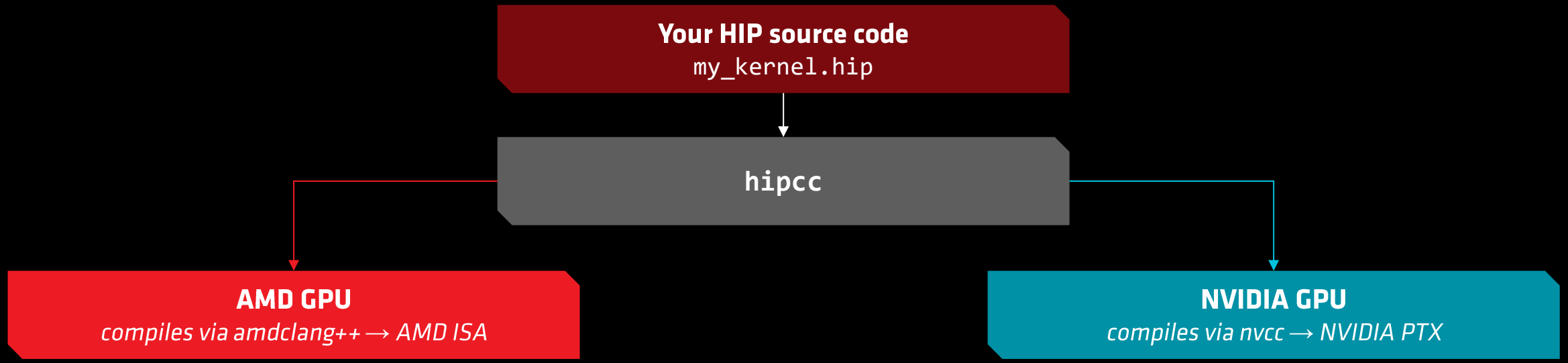
- Reminder: HIP – Heterogeneous-compute Interface for Portability
- Reminder: Thread Hierarchy and Mapping Threads to the Chip
- Host-Device Programming

AMD
together we advance_

Public

Reminder: HIP – Heterogeneous-compute Interface for Portability

Maintain a single source code base that can be compiled for AMD or NVIDIA GPUs.



SAME SOURCE

Portable source can target either platform; isolate platform-specific code when needed.

CUDA-LIKE SYNTAX

__global__, <<<grid,block>>>, threadIdx, blockIdx – all the familiar CUDA primitives.

CUDA PORTING PATH

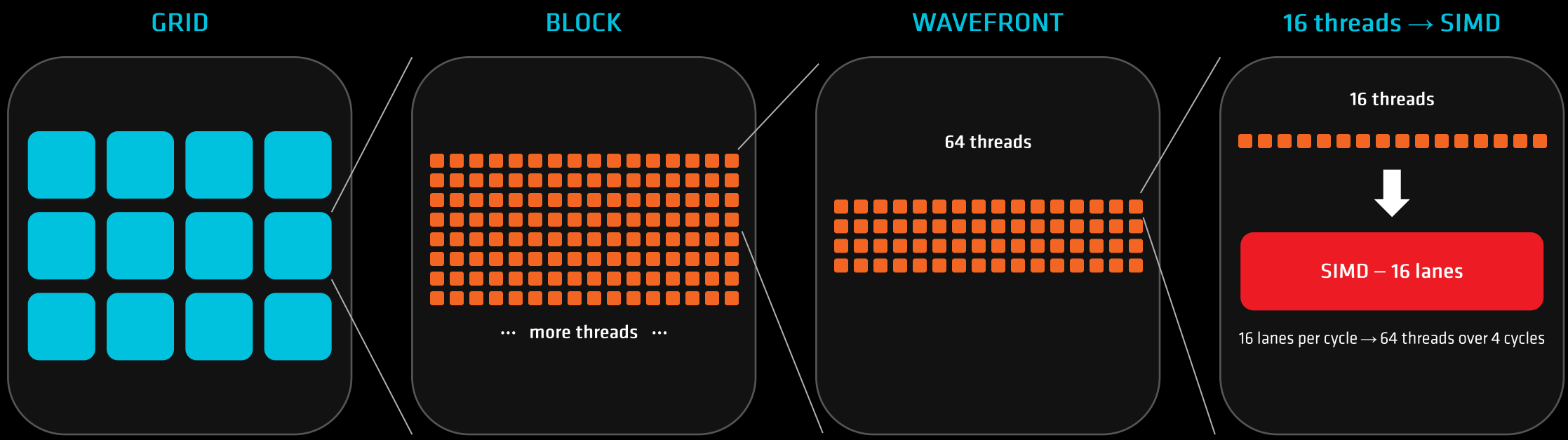
Run HIPIFY tools to translate standard CUDA API to HIP

AMD
together we advance_

Public

Reminder: Thread Hierarchy and Mapping Threads to the Chip

How the GPU actually runs your code.



GRID – array of blocks

BLOCK – up to 1024 threads

WAVEFRONT – 64 threads executed in lockstep on the hardware

SIMD – 16 lanes; 64 ⟶ 4 cycles

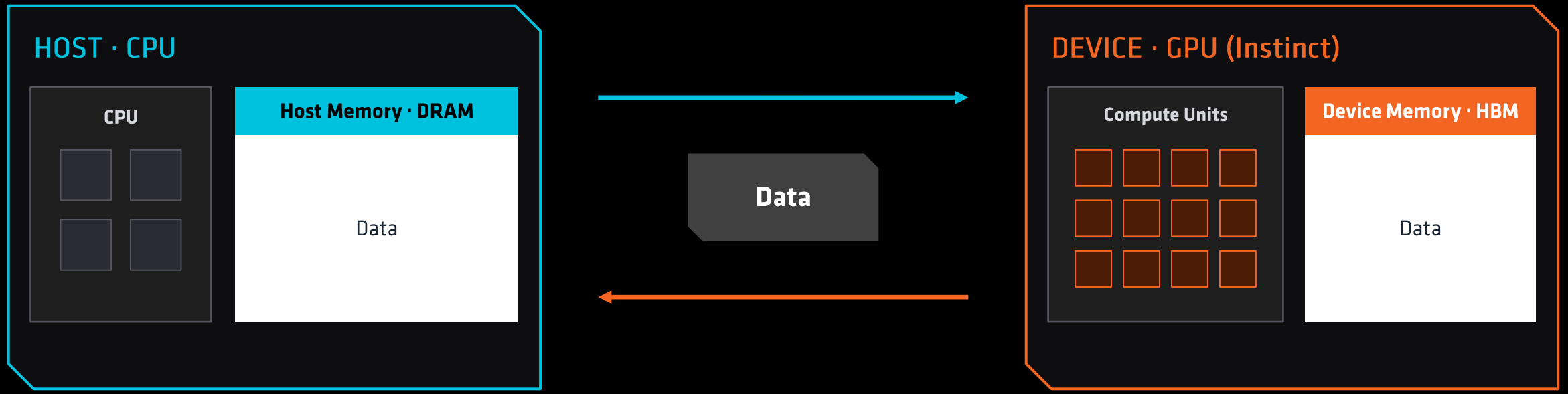
AMD
together we advance_

Public

Host-Device Programming

THEORY

Two separate processors, each with their own physical memories. Data must be passed between the processors using hipMemcpy to solve problems cooperatively.



SEPARATE ADDRESS SPACES

Host and device memory are separate. Keep host (h_X) and device (d_X) copies, and transfer data between them.

EXPLICIT DATA TRANSFER

Data does not move automatically. hipMemcpy explicitly transfers data between host and device memory when needed.

PERFECTLY PARALLEL

Each C[i] = A[i] + B[i] is independent. With N=2^20, about one million elements can run across about one million GPU threads.

AMD
together we advance_

Public

HIP Programming

The basics of HIP programming through a vector addition example.

- Vector Addition and the HIP API Calls it Requires
- HIP Vector Addition Code Walkthrough
- HIP Error Checking and Timing GPU Operations with HIP Events

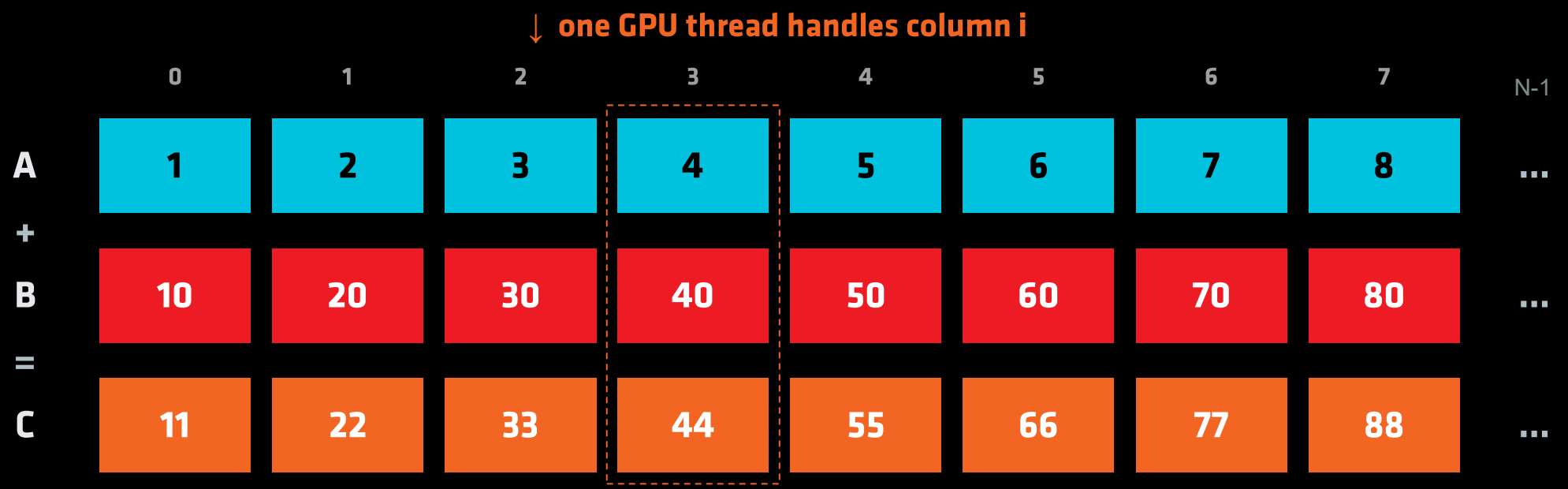
AMD
together we advance_

Public

Vector Addition – The Schema: C[i] = A[i] + B[i]

THEORY

Three vectors, one addition per element – and one GPU thread per element.



ONE THREAD, ONE ELEMENT

Thread i reads A[i] and B[i] and writes C[i]. Its index is i = blockDim.x·blockIdx.x + threadIdx.x.

FULLY INDEPENDENT

All N additions are independent, so the GPU can schedule them across many threads.

NO LOOP

The CPU loop over N elements becomes a kernel launch with one GPU thread per element; the GPU schedules and executes the threads in parallel.

AMD
together we advance_

Public

Vector Addition – Code Outline

THEORY

The host-device programming workflow for vector addition.

PURPOSE	SPECIFIC USE	HIP API / CODE PATTERN
Host (CPU) Memory Allocation	Allocate h_A, h_B, h_C in host memory.	`float *host_buf = new float[N];`
Host Buffer Initialization	Initialize h_A and h_B with input values.	`for (int i = 0; i < N; i++) { //set host array values }`
Device (GPU) Memory Allocation	Allocate d_A, d_B, d_C in device memory.	`hipMalloc(&device_buf, bytes);`
Host-to-Device Data Transfer	Copy h_A → d_A and h_B → d_B	`hipMemcpy(dst_buf, src_buf, bytes, hipMemcpyHostToDevice);`
Kernel Launch	Launch 1 GPU thread per output element.	`vec_add<<<(N+tpb-1)/tpb, tpb>>>(…); //tpb: threads/block`
Device-to-Host Data Transfer	Copy values of d_C (GPU results) → h_C	`hipMemcpy(dst_buf, src_buf, bytes, hipMemcpyDeviceToHost)`
Verify Results	Check values of h_C for correctness.	`for (int i = 0; i < N; i++) { //check results }`
Free Device and Host Memory	Deallocate d_{A,B,C} and h_{A,B,C}	`hipFree(device_buf), delete[] host_buf`

Explicit synchronization not needed after kernel launch since hipMemcpy is a blocking call.
Error checking omitted for clarity.

size_t bytes = N * sizeof(float);

AMD
together we advance_

Public

Example – Vector Addition

Walk through a HIP vector addition program.

PRACTICE

CODE CALLOUT

EXECUTION DOMAIN (HOST, DEVICE)

Describes the purpose of each code callout.

```
#include <hip/hip_runtime.h>
#include <stdio.h>

__global__ void vec_add(const float* A, const float* B,
                        float* C, int N) {

    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) C[i] = A[i] + B[i];
}

int main() {
    const int N  = 1 << 20;            // 1,048,576 elements
    size_t bytes = N * sizeof(float);  // 4 MiB per vector

    float *h_A = new float[N];
    float *h_B = new float[N];
    float *h_C = new float[N];

    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
        h_B[i] = i * 2.0f;
    }

    float *d_A, *d_B, *d_C;
    hipMalloc(&d_A, bytes);
    hipMalloc(&d_B, bytes);
    hipMalloc(&d_C, bytes);

    hipMemcpy(d_A, h_A, bytes, hipMemcpyHostToDevice);
    hipMemcpy(d_B, h_B, bytes, hipMemcpyHostToDevice);

    int thr_per_blk = 256;
    int blk_in_grid = (N + thr_per_blk - 1) / thr_per_blk;

    vec_add<<<blk_in_grid, thr_per_blk>>>(d_A, d_B, d_C, N);

    hipMemcpy(h_C, d_C, bytes, hipMemcpyDeviceToHost);

    int errs = 0;
    for (int i = 0; i < N; i++)
        if (h_C[i] != i * 3.0f) errs++;
    printf("Vector Add: N=%d, Errors=%d - %s\n",
            N, errs, errs ? "FAIL":"PASS");

    for (int i = 0; i < 5; i++) {
        printf("  C[%d] = %.1f\n", i, h_C[i]);
    }

    hipFree(d_A); hipFree(d_B); hipFree(d_C);
    delete[] h_A; delete[] h_B; delete[] h_C;

    return errs ? 1 : 0;
}
```

Explicit synchronization not needed after kernel launch since hipMemcpy is a blocking call.
Error checking omitted for clarity.

AMD
together we advance_

Public

Example – Vector Addition

Include header files.

PRACTICE

HIP RUNTIME HEADER

HOST AND DEVICE

Makes HIP available to the program – including memory management, data transfers, kernel launches, and GPU kernel constructs.

```
#include <hip/hip_runtime.h>
#include <stdio.h>

__global__ void vec_add(const float* A, const float* B,
                        float* C, int N) {

    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) C[i] = A[i] + B[i];
}

int main() {
    const int N  = 1 << 20;            // 1,048,576 elements
    size_t bytes = N * sizeof(float);  // 4 MiB per vector

    float *h_A = new float[N];
    float *h_B = new float[N];
    float *h_C = new float[N];

    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
        h_B[i] = i * 2.0f;
    }

    float *d_A, *d_B, *d_C;
    hipMalloc(&d_A, bytes);
    hipMalloc(&d_B, bytes);
    hipMalloc(&d_C, bytes);
    hipMemcpy(d_A, h_A, bytes, hipMemcpyHostToDevice);
    hipMemcpy(d_B, h_B, bytes, hipMemcpyHostToDevice);

    int thr_per_blk = 256;
    int blk_in_grid = (N + thr_per_blk - 1) / thr_per_blk;

    vec_add<<<blk_in_grid, thr_per_blk>>>(d_A, d_B, d_C, N);

    hipMemcpy(h_C, d_C, bytes, hipMemcpyDeviceToHost);

    int errs = 0;
    for (int i = 0; i < N; i++)
        if (h_C[i] != i * 3.0f) errs++;
    printf("Vector Add: N=%d, Errors=%d - %s\n",
           N, errs, errs ? "FAIL":"PASS");

    for (int i = 0; i < 5; i++) {
        printf("  C[%d] = %.1f\n", i, h_C[i]);
    }

    hipFree(d_A); hipFree(d_B); hipFree(d_C);
    delete[] h_A; delete[] h_B; delete[] h_C;

    return errs ? 1 : 0;
}
```


AMD
together we advance_

Public

Example – Vector Addition

Vector addition GPU Kernel.

PRACTICE

GPU KERNEL

DEVICE

A function executed in parallel by many GPU threads.

This kernel performs an element-wise addition of arrays A and B and stores the results in C.

We will describe the kernel in more detail following the walkthrough.

```
#include <hip/hip_runtime.h>
#include <stdio.h>

__global__ void vec_add(const float* A, const float* B,
                        float* C, int N) {

    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) C[i] = A[i] + B[i];
}

int main() {
    const int N  = 1 << 20;            // 1,048,576 elements
    size_t bytes = N * sizeof(float);  // 4 MiB per vector

    float *h_A = new float[N];
    float *h_B = new float[N];
    float *h_C = new float[N];

    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
        h_B[i] = i * 2.0f;
    }

    float *d_A, *d_B, *d_C;
    hipMalloc(&d_A, bytes);
    hipMalloc(&d_B, bytes);
    hipMalloc(&d_C, bytes);
```

```
    hipMemcpy(d_A, h_A, bytes, hipMemcpyHostToDevice);
    hipMemcpy(d_B, h_B, bytes, hipMemcpyHostToDevice);

    int thr_per_blk = 256;
    int blk_in_grid = (N + thr_per_blk - 1) / thr_per_blk;

    vec_add<<<blk_in_grid, thr_per_blk>>>(d_A, d_B, d_C, N);

    hipMemcpy(h_C, d_C, bytes, hipMemcpyDeviceToHost);

    int errs = 0;
    for (int i = 0; i < N; i++)
        if (h_C[i] != i * 3.0f) errs++;
    printf("Vector Add: N=%d, Errors=%d - %s\n",
           N, errs, errs ? "FAIL":"PASS");

    for (int i = 0; i < 5; i++) {
        printf("  C[%d] = %.1f\n", i, h_C[i]);
    }

    hipFree(d_A); hipFree(d_B); hipFree(d_C);
    delete[] h_A; delete[] h_B; delete[] h_C;

    return errs ? 1 : 0;
}
```


AMD
together we advance_

Public

Example – Vector Addition

The main function.

PRACTICE

MAIN FUNCTION

HOST

Host-side entry point that coordinates memory allocation, data transfers, kernel launch, verification, and cleanup.

```
#include <hip/hip_runtime.h>
#include <stdio.h>

__global__ void vec_add(const float* A, const float* B,
                        float* C, int N) {

    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) C[i] = A[i] + B[i];
}

int main() {
    const int N  = 1 << 20;            // 1,048,576 elements
    size_t bytes = N * sizeof(float);  // 4 MiB per vector

    float *h_A = new float[N];
    float *h_B = new float[N];
    float *h_C = new float[N];

    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
        h_B[i] = i * 2.0f;
    }

    float *d_A, *d_B, *d_C;
    hipMalloc(&d_A, bytes);
    hipMalloc(&d_B, bytes);
    hipMalloc(&d_C, bytes);
```

```
    hipMemcpy(d_A, h_A, bytes, hipMemcpyHostToDevice);
    hipMemcpy(d_B, h_B, bytes, hipMemcpyHostToDevice);

    int thr_per_blk = 256;
    int blk_in_grid = (N + thr_per_blk - 1) / thr_per_blk;

    vec_add<<<blk_in_grid, thr_per_blk>>>(d_A, d_B, d_C, N);

    hipMemcpy(h_C, d_C, bytes, hipMemcpyDeviceToHost);

    int errs = 0;
    for (int i = 0; i < N; i++)
        if (h_C[i] != i * 3.0f) errs++;
    printf("Vector Add: N=%d, Errors=%d - %s\n",
           N, errs, errs ? "FAIL":"PASS");

    for (int i = 0; i < 5; i++) {
        printf("  C[%d] = %.1f\n", i, h_C[i]);
    }

    hipFree(d_A); hipFree(d_B); hipFree(d_C);
    delete[] h_A; delete[] h_B; delete[] h_C;

    return errs ? 1 : 0;
}
```


AMD
together we advance_

Public

Example – Vector Addition

Set length and size of arrays.

PRACTICE

CONFIGURE ARRAYS

HOST

Set the number of elements in each array as well as their size in bytes.

```
#include <hip/hip_runtime.h>
#include <stdio.h>

__global__ void vec_add(const float* A, const float* B,
                        float* C, int N) {

    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) C[i] = A[i] + B[i];
}

int main() {
    const int N  = 1 << 20;            // 1,048,576 elements
    size_t bytes = N * sizeof(float);  // 4 MiB per vector

    float *h_A = new float[N];
    float *h_B = new float[N];
    float *h_C = new float[N];

    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
        h_B[i] = i * 2.0f;
    }

    float *d_A, *d_B, *d_C;
    hipMalloc(&d_A, bytes);
    hipMalloc(&d_B, bytes);
    hipMalloc(&d_C, bytes);
```

```
    hipMemcpy(d_A, h_A, bytes, hipMemcpyHostToDevice);
    hipMemcpy(d_B, h_B, bytes, hipMemcpyHostToDevice);

    int thr_per_blk = 256;
    int blk_in_grid = (N + thr_per_blk - 1) / thr_per_blk;

    vec_add<<<blk_in_grid, thr_per_blk>>>(d_A, d_B, d_C, N);

    hipMemcpy(h_C, d_C, bytes, hipMemcpyDeviceToHost);

    int errs = 0;
    for (int i = 0; i < N; i++)
        if (h_C[i] != i * 3.0f) errs++;
    printf("Vector Add: N=%d, Errors=%d - %s\n",
           N, errs, errs ? "FAIL":"PASS");

    for (int i = 0; i < 5; i++) {
        printf("  C[%d] = %.1f\n", i, h_C[i]);
    }

    hipFree(d_A); hipFree(d_B); hipFree(d_C);
    delete[] h_A; delete[] h_B; delete[] h_C;

    return errs ? 1 : 0;
}
```


AMD
together we advance_

Public

Example – Vector Addition

Allocate memory for host arrays.

PRACTICE

HOST ARRAY MEMORY ALLOCATION

HOST

Allocate memory buffers for arrays h_A, h_B, and h_C on the host (CPU).

```
#include <hip/hip_runtime.h>
#include <stdio.h>

__global__ void vec_add(const float* A, const float* B,
                        float* C, int N) {

    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) C[i] = A[i] + B[i];
}

int main() {
    const int N  = 1 << 20;            // 1,048,576 elements
    size_t bytes = N * sizeof(float);  // 4 MiB per vector

    float *h_A = new float[N];
    float *h_B = new float[N];
    float *h_C = new float[N];

    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
        h_B[i] = i * 2.0f;
    }

    float *d_A, *d_B, *d_C;
    hipMalloc(&d_A, bytes);
    hipMalloc(&d_B, bytes);
    hipMalloc(&d_C, bytes);
```

```
    hipMemcpy(d_A, h_A, bytes, hipMemcpyHostToDevice);
    hipMemcpy(d_B, h_B, bytes, hipMemcpyHostToDevice);

    int thr_per_blk = 256;
    int blk_in_grid = (N + thr_per_blk - 1) / thr_per_blk;

    vec_add<<<blk_in_grid, thr_per_blk>>>(d_A, d_B, d_C, N);

    hipMemcpy(h_C, d_C, bytes, hipMemcpyDeviceToHost);

    int errs = 0;
    for (int i = 0; i < N; i++)
        if (h_C[i] != i * 3.0f) errs++;
    printf("Vector Add: N=%d, Errors=%d - %s\n",
           N, errs, errs ? "FAIL":"PASS");

    for (int i = 0; i < 5; i++) {
        printf("  C[%d] = %.1f\n", i, h_C[i]);
    }

    hipFree(d_A); hipFree(d_B); hipFree(d_C);
    delete[] h_A; delete[] h_B; delete[] h_C;

    return errs ? 1 : 0;
}
```


AMD
together we advance_

Public

Example – Vector Addition

Set initial values for host arrays.

PRACTICE

INITIALIZE HOST ARRAY VALUES

HOST

Initialize the values of host arrays h_A and h_B.

```
#include <hip/hip_runtime.h>
#include <stdio.h>

__global__ void vec_add(const float* A, const float* B,
                        float* C, int N) {

    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) C[i] = A[i] + B[i];
}

int main() {
    const int N  = 1 << 20;            // 1,048,576 elements
    size_t bytes = N * sizeof(float);  // 4 MiB per vector

    float *h_A = new float[N];
    float *h_B = new float[N];
    float *h_C = new float[N];

    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
        h_B[i] = i * 2.0f;
    }

    float *d_A, *d_B, *d_C;
    hipMalloc(&d_A, bytes);
    hipMalloc(&d_B, bytes);
    hipMalloc(&d_C, bytes);
```

```
    hipMemcpy(d_A, h_A, bytes, hipMemcpyHostToDevice);
    hipMemcpy(d_B, h_B, bytes, hipMemcpyHostToDevice);

    int thr_per_blk = 256;
    int blk_in_grid = (N + thr_per_blk - 1) / thr_per_blk;

    vec_add<<<blk_in_grid, thr_per_blk>>>(d_A, d_B, d_C, N);

    hipMemcpy(h_C, d_C, bytes, hipMemcpyDeviceToHost);

    int errs = 0;
    for (int i = 0; i < N; i++)
        if (h_C[i] != i * 3.0f) errs++;
    printf("Vector Add: N=%d, Errors=%d - %s\n",
           N, errs, errs ? "FAIL":"PASS");

    for (int i = 0; i < 5; i++) {
        printf("  C[%d] = %.1f\n", i, h_C[i]);
    }

    hipFree(d_A); hipFree(d_B); hipFree(d_C);
    delete[] h_A; delete[] h_B; delete[] h_C;

    return errs ? 1 : 0;
}
```


AMD
together we advance_

Public

Example – Vector Addition

Allocate memory for device arrays.

PRACTICE

DEVICE ARRAY MEMORY ALLOCATION

HOST

Allocate memory buffers for arrays d_A, d_B, and d_C on the device (GPU).

hipMalloc allocates memory on the device (GPU); it is executed from the host (CPU) through the HIP runtime API.

```
#include <hip/hip_runtime.h>
#include <stdio.h>

__global__ void vec_add(const float* A, const float* B,
                        float* C, int N) {

    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) C[i] = A[i] + B[i];
}

int main() {
    const int N  = 1 << 20;            // 1,048,576 elements
    size_t bytes = N * sizeof(float);  // 4 MiB per vector

    float *h_A = new float[N];
    float *h_B = new float[N];
    float *h_C = new float[N];

    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
        h_B[i] = i * 2.0f;
    }

    float *d_A, *d_B, *d_C;
    hipMalloc(&d_A, bytes);
    hipMalloc(&d_B, bytes);
    hipMalloc(&d_C, bytes);
```

```
    hipMemcpy(d_A, h_A, bytes, hipMemcpyHostToDevice);
    hipMemcpy(d_B, h_B, bytes, hipMemcpyHostToDevice);

    int thr_per_blk = 256;
    int blk_in_grid = (N + thr_per_blk - 1) / thr_per_blk;

    vec_add<<<blk_in_grid, thr_per_blk>>>(d_A, d_B, d_C, N);

    hipMemcpy(h_C, d_C, bytes, hipMemcpyDeviceToHost);

    int errs = 0;
    for (int i = 0; i < N; i++)
        if (h_C[i] != i * 3.0f) errs++;
    printf("Vector Add: N=%d, Errors=%d - %s\n",
           N, errs, errs ? "FAIL":"PASS");

    for (int i = 0; i < 5; i++) {
        printf("  C[%d] = %.1f\n", i, h_C[i]);
    }

    hipFree(d_A); hipFree(d_B); hipFree(d_C);
    delete[] h_A; delete[] h_B; delete[] h_C;

    return errs ? 1 : 0;
}
```


AMD
together we advance_

Public

Example – Vector Addition

Copy data from host to device.

PRACTICE

MEMORY COPY FROM HOST TO DEVICE

HOST

Copy the initialized data in the host buffers (arrays) h_A and h_B to the device buffers (arrays) d_A and d_B.

hipMemcpy transfers data between the host and device memory; it is executed from the host through the HIP runtime API.

```
#include <hip/hip_runtime.h>
#include <stdio.h>

__global__ void vec_add(const float* A, const float* B,
                        float* C, int N) {

    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) C[i] = A[i] + B[i];
}

int main() {
    const int N  = 1 << 20;            // 1,048,576 elements
    size_t bytes = N * sizeof(float);  // 4 MiB per vector

    float *h_A = new float[N];
    float *h_B = new float[N];
    float *h_C = new float[N];

    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
        h_B[i] = i * 2.0f;
    }

    float *d_A, *d_B, *d_C;
    hipMalloc(&d_A, bytes);
    hipMalloc(&d_B, bytes);
    hipMalloc(&d_C, bytes);
```

```
    hipMemcpy(d_A, h_A, bytes, hipMemcpyHostToDevice);
    hipMemcpy(d_B, h_B, bytes, hipMemcpyHostToDevice);

    int thr_per_blk = 256;
    int blk_in_grid = (N + thr_per_blk - 1) / thr_per_blk;

    vec_add<<<blk_in_grid, thr_per_blk>>>(d_A, d_B, d_C, N);

    hipMemcpy(h_C, d_C, bytes, hipMemcpyDeviceToHost);

    int errs = 0;
    for (int i = 0; i < N; i++)
        if (h_C[i] != i * 3.0f) errs++;
    printf("Vector Add: N=%d, Errors=%d - %s\n",
           N, errs, errs ? "FAIL":"PASS");

    for (int i = 0; i < 5; i++) {
        printf("  C[%d] = %.1f\n", i, h_C[i]);
    }

    hipFree(d_A); hipFree(d_B); hipFree(d_C);
    delete[] h_A; delete[] h_B; delete[] h_C;

    return errs ? 1 : 0;
}
```


AMD
together we advance_

Public

Example – Vector Addition

Set kernel configuration parameters and launch kernel.

PRACTICE

CONFIGURE AND LAUNCH KERNEL

HOST

Set the number of threads in each block and total number of blocks in the grid; then launch the kernel on the GPU.

The host submits the kernel and its grid config for execution on the GPU.

We will describe kernel configuration and launch in more detail following the walkthrough.

```
#include <hip/hip_runtime.h>
#include <stdio.h>

__global__ void vec_add(const float* A, const float* B,
                        float* C, int N) {

    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) C[i] = A[i] + B[i];
}

int main() {
    const int N  = 1 << 20;            // 1,048,576 elements
    size_t bytes = N * sizeof(float);  // 4 MiB per vector

    float *h_A = new float[N];
    float *h_B = new float[N];
    float *h_C = new float[N];

    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
        h_B[i] = i * 2.0f;
    }

    float *d_A, *d_B, *d_C;
    hipMalloc(&d_A, bytes);
    hipMalloc(&d_B, bytes);
    hipMalloc(&d_C, bytes);
```

```
    hipMemcpy(d_A, h_A, bytes, hipMemcpyHostToDevice);
    hipMemcpy(d_B, h_B, bytes, hipMemcpyHostToDevice);

    int thr_per_blk = 256;
    int blk_in_grid = (N + thr_per_blk - 1) / thr_per_blk;

    vec_add<<<blk_in_grid, thr_per_blk>>>(d_A, d_B, d_C, N);

    hipMemcpy(h_C, d_C, bytes, hipMemcpyDeviceToHost);

    int errs = 0;
    for (int i = 0; i < N; i++)
        if (h_C[i] != i * 3.0f) errs++;
    printf("Vector Add: N=%d, Errors=%d - %s\n",
           N, errs, errs ? "FAIL":"PASS");

    for (int i = 0; i < 5; i++) {
        printf("  C[%d] = %.1f\n", i, h_C[i]);
    }

    hipFree(d_A); hipFree(d_B); hipFree(d_C);
    delete[] h_A; delete[] h_B; delete[] h_C;

    return errs ? 1 : 0;
}
```


AMD
together we advance_

Public

Example – Vector Addition

Copy data from device to host.

PRACTICE

MEMORY COPY FROM DEVICE TO HOST

HOST

Copy the resulting data in the device buffer (array) d_C to the host buffer (array) h_C.

hipMemcpy is a blocking call: it waits for earlier GPU work to complete and for the copy to finish before continuing on the host. No explicit synchronization is needed here.

```
#include <hip/hip_runtime.h>
#include <stdio.h>

__global__ void vec_add(const float* A, const float* B,
                        float* C, int N) {

    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) C[i] = A[i] + B[i];
}

int main() {
    const int N  = 1 << 20;            // 1,048,576 elements
    size_t bytes = N * sizeof(float);  // 4 MiB per vector

    float *h_A = new float[N];
    float *h_B = new float[N];
    float *h_C = new float[N];

    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
        h_B[i] = i * 2.0f;
    }

    float *d_A, *d_B, *d_C;
    hipMalloc(&d_A, bytes);
    hipMalloc(&d_B, bytes);
    hipMalloc(&d_C, bytes);
```

```
    hipMemcpy(d_A, h_A, bytes, hipMemcpyHostToDevice);
    hipMemcpy(d_B, h_B, bytes, hipMemcpyHostToDevice);

    int thr_per_blk = 256;
    int blk_in_grid = (N + thr_per_blk - 1) / thr_per_blk;

    vec_add<<<blk_in_grid, thr_per_blk>>>(d_A, d_B, d_C, N);

    hipMemcpy(h_C, d_C, bytes, hipMemcpyDeviceToHost);

    int errs = 0;
    for (int i = 0; i < N; i++)
        if (h_C[i] != i * 3.0f) errs++;
    printf("Vector Add: N=%d, Errors=%d - %s\n",
           N, errs, errs ? "FAIL":"PASS");

    for (int i = 0; i < 5; i++) {
        printf("  C[%d] = %.1f\n", i, h_C[i]);
    }

    hipFree(d_A); hipFree(d_B); hipFree(d_C);
    delete[] h_A; delete[] h_B; delete[] h_C;

    return errs ? 1 : 0;
}
```


AMD
together we advance_

Public

Example – Vector Addition

Check results for correctness.

PRACTICE

VERIFY RESULTS

HOST

Verify that the results obtained from the GPU are correct, and print a subset of the results.

```
#include <hip/hip_runtime.h>
#include <stdio.h>

__global__ void vec_add(const float* A, const float* B,
                        float* C, int N) {

    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) C[i] = A[i] + B[i];
}

int main() {
    const int N  = 1 << 20;            // 1,048,576 elements
    size_t bytes = N * sizeof(float);  // 4 MiB per vector

    float *h_A = new float[N];
    float *h_B = new float[N];
    float *h_C = new float[N];

    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
        h_B[i] = i * 2.0f;
    }

    float *d_A, *d_B, *d_C;
    hipMalloc(&d_A, bytes);
    hipMalloc(&d_B, bytes);
    hipMalloc(&d_C, bytes);
```

```
    hipMemcpy(d_A, h_A, bytes, hipMemcpyHostToDevice);
    hipMemcpy(d_B, h_B, bytes, hipMemcpyHostToDevice);

    int thr_per_blk = 256;
    int blk_in_grid = (N + thr_per_blk - 1) / thr_per_blk;

    vec_add<<<blk_in_grid, thr_per_blk>>>(d_A, d_B, d_C, N);

    hipMemcpy(h_C, d_C, bytes, hipMemcpyDeviceToHost);

    int errs = 0;
    for (int i = 0; i < N; i++)
        if (h_C[i] != i * 3.0f) errs++;
    printf("Vector Add: N=%d, Errors=%d - %s\n",
            N, errs, errs ? "FAIL":"PASS");

    for (int i = 0; i < 5; i++) {
        printf("  C[%d] = %.1f\n", i, h_C[i]);
    }

    hipFree(d_A); hipFree(d_B); hipFree(d_C);
    delete[] h_A; delete[] h_B; delete[] h_C;

    return errs ? 1 : 0;
}
```


AMD
together we advance_

Public

Example – Vector Addition

Free device and host memory buffers.

PRACTICE

FREE MEMORY

HOST

Free (deallocate) the host and device memory buffers.

```
#include <hip/hip_runtime.h>
#include <stdio.h>

__global__ void vec_add(const float* A, const float* B,
                        float* C, int N) {

    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) C[i] = A[i] + B[i];
}

int main() {
    const int N  = 1 << 20;            // 1,048,576 elements
    size_t bytes = N * sizeof(float);  // 4 MiB per vector

    float *h_A = new float[N];
    float *h_B = new float[N];
    float *h_C = new float[N];

    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
        h_B[i] = i * 2.0f;
    }

    float *d_A, *d_B, *d_C;
    hipMalloc(&d_A, bytes);
    hipMalloc(&d_B, bytes);
    hipMalloc(&d_C, bytes);
    hipMemcpy(d_A, h_A, bytes, hipMemcpyHostToDevice);
    hipMemcpy(d_B, h_B, bytes, hipMemcpyHostToDevice);

    int thr_per_blk = 256;
    int blk_in_grid = (N + thr_per_blk - 1) / thr_per_blk;

    vec_add<<<blk_in_grid, thr_per_blk>>>(d_A, d_B, d_C, N);

    hipMemcpy(h_C, d_C, bytes, hipMemcpyDeviceToHost);

    int errs = 0;
    for (int i = 0; i < N; i++)
        if (h_C[i] != i * 3.0f) errs++;
    printf("Vector Add: N=%d, Errors=%d - %s\n",
           N, errs, errs ? "FAIL":"PASS");

    for (int i = 0; i < 5; i++) {
        printf("  C[%d] = %.1f\n", i, h_C[i]);
    }

    hipFree(d_A); hipFree(d_B); hipFree(d_C);
    delete[] h_A; delete[] h_B; delete[] h_C;

    return errs ? 1 : 0;
}
```


AMD
together we advance_

Public

Example – Vector Addition

Return and exit.

PRACTICE

RETURN AND EXIT

HOST

Return 0 or 1 (if errors) and exit.

```
#include <hip/hip_runtime.h>
#include <stdio.h>

__global__ void vec_add(const float* A, const float* B,
                        float* C, int N) {

    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) C[i] = A[i] + B[i];
}

int main() {
    const int N  = 1 << 20;            // 1,048,576 elements
    size_t bytes = N * sizeof(float);  // 4 MiB per vector

    float *h_A = new float[N];
    float *h_B = new float[N];
    float *h_C = new float[N];

    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
        h_B[i] = i * 2.0f;
    }

    float *d_A, *d_B, *d_C;
    hipMalloc(&d_A, bytes);
    hipMalloc(&d_B, bytes);
    hipMalloc(&d_C, bytes);
```

```
    hipMemcpy(d_A, h_A, bytes, hipMemcpyHostToDevice);
    hipMemcpy(d_B, h_B, bytes, hipMemcpyHostToDevice);

    int thr_per_blk = 256;
    int blk_in_grid = (N + thr_per_blk - 1) / thr_per_blk;

    vec_add<<<blk_in_grid, thr_per_blk>>>(d_A, d_B, d_C, N);

    hipMemcpy(h_C, d_C, bytes, hipMemcpyDeviceToHost);

    int errs = 0;
    for (int i = 0; i < N; i++)
        if (h_C[i] != i * 3.0f) errs++;
    printf("Vector Add: N=%d, Errors=%d - %s\n",
           N, errs, errs ? "FAIL":"PASS");

    for (int i = 0; i < 5; i++) {
        printf("  C[%d] = %.1f\n", i, h_C[i]);
    }

    hipFree(d_A); hipFree(d_B); hipFree(d_C);
    delete[] h_A; delete[] h_B; delete[] h_C;

    return errs ? 1 : 0;
}
```


AMD
together we advance_

Public

Example – Vector Addition Kernel and Kernel Launch

PRACTICE

Walk through the GPU kernel and kernel launch.

GPU KERNEL AND KERNEL LAUNCH

HOST AND DEVICE

The kernel is configured and launched from the host (CPU) and executed on the device (GPU).

```
#include <hip/hip_runtime.h>
#include <stdio.h>

__global__ void vec_add(const float* A, const float* B,
                        float* C, int N) {

    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) C[i] = A[i] + B[i];
}

int main() {
    const int N  = 1 << 20;            // 1,048,576 elements
    size_t bytes = N * sizeof(float);  // 4 MiB per vector

    float *h_A = new float[N];
    float *h_B = new float[N];
    float *h_C = new float[N];

    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
        h_B[i] = i * 2.0f;
    }

    float *d_A, *d_B, *d_C;
    hipMalloc(&d_A, bytes);
    hipMalloc(&d_B, bytes);
    hipMalloc(&d_C, bytes);
```

```
    hipMemcpy(d_A, h_A, bytes, hipMemcpyHostToDevice);
    hipMemcpy(d_B, h_B, bytes, hipMemcpyHostToDevice);

    int thr_per_blk = 256;
    int blk_in_grid = (N + thr_per_blk - 1) / thr_per_blk;

    vec_add<<<blk_in_grid, thr_per_blk>>>(d_A, d_B, d_C, N);

    hipMemcpy(h_C, d_C, bytes, hipMemcpyDeviceToHost);

    int errs = 0;
    for (int i = 0; i < N; i++)
        if (h_C[i] != i * 3.0f) errs++;
    printf("Vector Add: N=%d, Errors=%d - %s\n",
           N, errs, errs ? "FAIL":"PASS");

    for (int i = 0; i < 5; i++) {
        printf("  C[%d] = %.1f\n", i, h_C[i]);
    }

    hipFree(d_A); hipFree(d_B); hipFree(d_C);
    delete[] h_A; delete[] h_B; delete[] h_C;

    return errs ? 1 : 0;
}
```


AMD
together we advance_

Public

Example – Vector Addition Kernel

Understand the GPU kernel declaration components.

PRACTICE

GPU KERNEL DECLARATION

DEVICE

Defines the kernel's name, execution type, and input parameters.

__global__: a GPU function launched from host

Kernels are void type

vec_add is the name we chose

A, B, C are GPU mem buffers allocated with hipMalloc

```
#include <hip/hip_runtime.h>
#include <stdio.h>

__global__ void vec_add(const float* A, const float* B,
                        float* C, int N) {

    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) C[i] = A[i] + B[i];
}

int main() {
    const int N  = 1 << 20;            // 1,048,576 elements
    size_t bytes = N * sizeof(float);  // 4 MiB per vector

    float *h_A = new float[N];
    float *h_B = new float[N];
    float *h_C = new float[N];

    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
        h_B[i] = i * 2.0f;
    }

    float *d_A, *d_B, *d_C;
    hipMalloc(&d_A, bytes);
    hipMalloc(&d_B, bytes);
    hipMalloc(&d_C, bytes);
```

```
    hipMemcpy(d_A, h_A, bytes, hipMemcpyHostToDevice);
    hipMemcpy(d_B, h_B, bytes, hipMemcpyHostToDevice);

    int thr_per_blk = 256;
    int blk_in_grid = (N + thr_per_blk - 1) / thr_per_blk;

    vec_add<<<blk_in_grid, thr_per_blk>>>(d_A, d_B, d_C, N);

    hipMemcpy(h_C, d_C, bytes, hipMemcpyDeviceToHost);

    int errs = 0;
    for (int i = 0; i < N; i++)
        if (h_C[i] != i * 3.0f) errs++;
    printf("Vector Add: N=%d, Errors=%d - %s\n",
            N, errs, errs ? "FAIL":"PASS");

    for (int i = 0; i < 5; i++) {
        printf("  C[%d] = %.1f\n", i, h_C[i]);
    }

    hipFree(d_A); hipFree(d_B); hipFree(d_C);
    delete[] h_A; delete[] h_B; delete[] h_C;

    return errs ? 1 : 0;
}
```


AMD
together we advance_

Public

Example – Vector Addition Kernel

PRACTICE

Global thread index allows each GPU thread to calculate which data element it should process.

GLOBAL THREAD INDEX

DEVICE

Defines a thread's global ID within the grid from built-ins.

blockDim: number of threads per block.

blockIdx: index of a block within the grid.

threadIdx: local index of a thread within a block.

Each is a dim3 with .x, .y, and .z components.

```
#include <hip/hip_runtime.h>
#include <stdio.h>

__global__ void vec_add(const float* A, const float* B,
                        float* C, int N) {

    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) C[i] = A[i] + B[i];
}

int main() {
    const int N  = 1 << 20;            // 1,048,576 elements
    size_t bytes = N * sizeof(float);  // 4 MiB per vector

    float *h_A = new float[N];
    float *h_B = new float[N];
    float *h_C = new float[N];

    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
        h_B[i] = i * 2.0f;
    }

    float *d_A, *d_B, *d_C;
    hipMalloc(&d_A, bytes);
    hipMalloc(&d_B, bytes);
    hipMalloc(&d_C, bytes);
```

```
    hipMemcpy(d_A, h_A, bytes, hipMemcpyHostToDevice);
    hipMemcpy(d_B, h_B, bytes, hipMemcpyHostToDevice);

    int thr_per_blk = 256;
    int blk_in_grid = (N + thr_per_blk - 1) / thr_per_blk;

    vec_add<<<blk_in_grid, thr_per_blk>>>(d_A, d_B, d_C, N);

    hipMemcpy(h_C, d_C, bytes, hipMemcpyDeviceToHost);

    int errs = 0;
    for (int i = 0; i < N; i++)
        if (h_C[i] != i * 3.0f) errs++;
    printf("Vector Add: N=%d, Errors=%d - %s\n",
           N, errs, errs ? "FAIL":"PASS");

    for (int i = 0; i < 5; i++) {
        printf("  C[%d] = %.1f\n", i, h_C[i]);
    }

    hipFree(d_A); hipFree(d_B); hipFree(d_C);
    delete[] h_A; delete[] h_B; delete[] h_C;

    return errs ? 1 : 0;
}
```


AMD
together we advance_

Public

Example – Vector Addition Kernel

Global thread ID example.

PRACTICE

GLOBAL THREAD INDEX

DEVICE

Defines a thread's global ID within the grid from built-ins.

blockDim: number of threads per block.

blockIdx: index of a block within the grid.

threadIdx: local index of a thread within a block.

Each is a dim3 with .x, .y, and .z components.

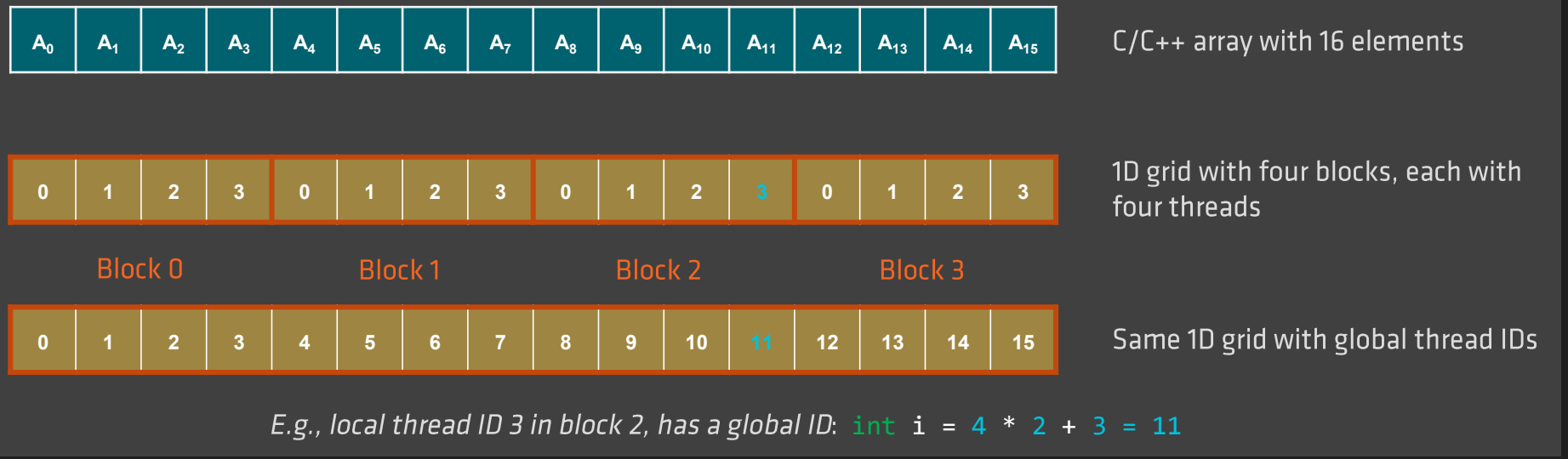
```
#include <hip/hip_runtime.h>
#include <stdio.h>

__global__ void vec_add(const float* A, const float* B,
                        float* C, int N) {

    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) C[i] = A[i] + B[i];
}
```

For simplicity, assume:

```
int N = 16;
int thr_per_blk = 4;
int blk_in_grid = (N + thr_per_blk - 1) / thr_per_blk; (= 4)
```



E.g., local thread ID 3 in block 2, has a global ID: `int i = 4 * 2 + 3 = 11`

AMD
together we advance_

Public

Example – Vector Addition Kernel

Bounds check and element-wise addition.

PRACTICE

BOUNDS CHECK AND ADDITION

DEVICE

Checks that the thread's index is within the array, then adds the corresponding elements of A and B and stores the result in C.

```
#include <hip/hip_runtime.h>
#include <stdio.h>

__global__ void vec_add(const float* A, const float* B,
                        float* C, int N) {

    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) C[i] = A[i] + B[i];
}

int main() {
    const int N  = 1 << 20;            // 1,048,576 elements
    size_t bytes = N * sizeof(float);  // 4 MiB per vector

    float *h_A = new float[N];
    float *h_B = new float[N];
    float *h_C = new float[N];

    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
        h_B[i] = i * 2.0f;
    }

    float *d_A, *d_B, *d_C;
    hipMalloc(&d_A, bytes);
    hipMalloc(&d_B, bytes);
    hipMalloc(&d_C, bytes);
```

```
    hipMemcpy(d_A, h_A, bytes, hipMemcpyHostToDevice);
    hipMemcpy(d_B, h_B, bytes, hipMemcpyHostToDevice);

    int thr_per_blk = 256;
    int blk_in_grid = (N + thr_per_blk - 1) / thr_per_blk;

    vec_add<<<blk_in_grid, thr_per_blk>>>(d_A, d_B, d_C, N);

    hipMemcpy(h_C, d_C, bytes, hipMemcpyDeviceToHost);

    int errs = 0;
    for (int i = 0; i < N; i++)
        if (h_C[i] != i * 3.0f) errs++;
    printf("Vector Add: N=%d, Errors=%d - %s\n",
           N, errs, errs ? "FAIL":"PASS");

    for (int i = 0; i < 5; i++) {
        printf("  C[%d] = %.1f\n", i, h_C[i]);
    }

    hipFree(d_A); hipFree(d_B); hipFree(d_C);
    delete[] h_A; delete[] h_B; delete[] h_C;

    return errs ? 1 : 0;
}
```


AMD
together we advance_

Public

Example – Vector Addition Kernel

Bounds check example.

PRACTICE

BOUNDS CHECK AND ADDITION

DEVICE

Checks that the thread's index is within the array, then adds the corresponding elements of A and B and stores the result in C.

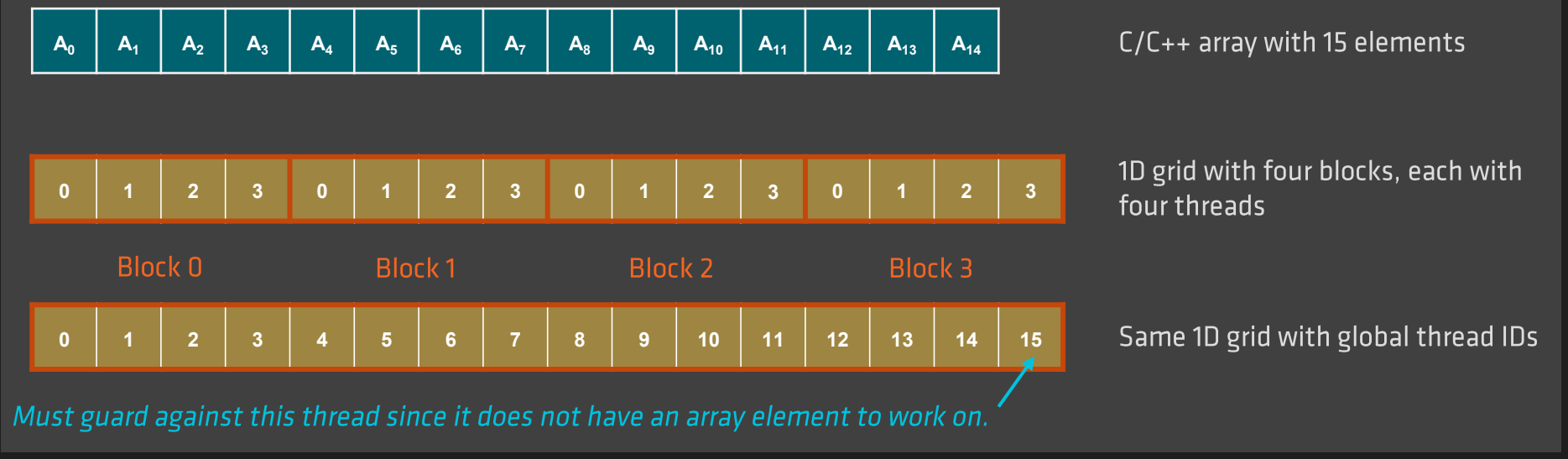
```
#include <hip/hip_runtime.h>
#include <stdio.h>

__global__ void vec_add(const float* A, const float* B,
                        float* C, int N) {

    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) C[i] = A[i] + B[i];
}
```

Now assume array size is only 15: *still 4 due to ceiling*

```
int N = 15;
int thr_per_blk = 4;
int blk_in_grid = (N + thr_per_blk - 1) / thr_per_blk; (= 4)
```



AMD
together we advance_

Public

Example – Vector Addition Kernel Launch

Set kernel configuration parameters.

PRACTICE

KERNEL LAUNCH CONFIG

HOST

Sets 256 threads per block, then calculates the number of blocks needed to provide at least one thread for every vector element.

Adding threads_per_block - 1 before integer division rounds up when N is not evenly divisible by the block size.

```
#include <hip/hip_runtime.h>
#include <stdio.h>

__global__ void vec_add(const float* A, const float* B,
                        float* C, int N) {

    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) C[i] = A[i] + B[i];
}

int main() {
    const int N  = 1 << 20;            // 1,048,576 elements
    size_t bytes = N * sizeof(float);  // 4 MiB per vector

    float *h_A = new float[N];
    float *h_B = new float[N];
    float *h_C = new float[N];

    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
        h_B[i] = i * 2.0f;
    }

    float *d_A, *d_B, *d_C;
    hipMalloc(&d_A, bytes);
    hipMalloc(&d_B, bytes);
    hipMalloc(&d_C, bytes);
```

```
    hipMemcpy(d_A, h_A, bytes, hipMemcpyHostToDevice);
    hipMemcpy(d_B, h_B, bytes, hipMemcpyHostToDevice);

    int thr_per_blk = 256;
    int blk_in_grid = (N + thr_per_blk - 1) / thr_per_blk;

    vec_add<<<blk_in_grid, thr_per_blk>>>(d_A, d_B, d_C, N);

    hipMemcpy(h_C, d_C, bytes, hipMemcpyDeviceToHost);

    int errs = 0;
    for (int i = 0; i < N; i++)
        if (h_C[i] != i * 3.0f) errs++;
    printf("Vector Add: N=%d, Errors=%d - %s\n",
           N, errs, errs ? "FAIL":"PASS");

    for (int i = 0; i < 5; i++) {
        printf("  C[%d] = %.1f\n", i, h_C[i]);
    }

    hipFree(d_A); hipFree(d_B); hipFree(d_C);
    delete[] h_A; delete[] h_B; delete[] h_C;

    return errs ? 1 : 0;
}
```


AMD
together we advance_

Public

Example – Vector Addition Kernel Launch

PRACTICE

Launch kernel to be executed on the GPU.

KERNEL LAUNCH CONFIG

HOST

Launch the kernel asynchronously from host onto the GPU.

kernel<<<>>> takes 2 required and 2 optional parameters.

Required: Number of blocks within the grid and number of threads per block.

Optional: Number of bytes of dynamic shared memory and stream ID.

```
#include <hip/hip_runtime.h>
#include <stdio.h>

__global__ void vec_add(const float* A, const float* B,
                        float* C, int N) {

    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) C[i] = A[i] + B[i];
}

int main() {
    const int N  = 1 << 20;            // 1,048,576 elements
    size_t bytes = N * sizeof(float);  // 4 MiB per vector

    float *h_A = new float[N];
    float *h_B = new float[N];
    float *h_C = new float[N];

    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
        h_B[i] = i * 2.0f;
    }

    float *d_A, *d_B, *d_C;
    hipMalloc(&d_A, bytes);
    hipMalloc(&d_B, bytes);
    hipMalloc(&d_C, bytes);
```

```
    hipMemcpy(d_A, h_A, bytes, hipMemcpyHostToDevice);
    hipMemcpy(d_B, h_B, bytes, hipMemcpyHostToDevice);

    int thr_per_blk = 256;
    int blk_in_grid = (N + thr_per_blk - 1) / thr_per_blk;

    vec_add<<<blk_in_grid, thr_per_blk>>>(d_A, d_B, d_C, N);

    hipMemcpy(h_C, d_C, bytes, hipMemcpyDeviceToHost);

    int errs = 0;
    for (int i = 0; i < N; i++)
        if (h_C[i] != i * 3.0f) errs++;
    printf("Vector Add: N=%d, Errors=%d - %s\n",
           N, errs, errs ? "FAIL":"PASS");

    for (int i = 0; i < 5; i++) {
        printf("  C[%d] = %.1f\n", i, h_C[i]);
    }

    hipFree(d_A); hipFree(d_B); hipFree(d_C);
    delete[] h_A; delete[] h_B; delete[] h_C;

    return errs ? 1 : 0;
}
```


AMD
together we advance_

Public

Example – Vector Addition Kernel Launch

Generalized form of a kernel launch.

PRACTICE

KERNEL LAUNCH CONFIG

HOST

Launch the kernel asynchronously from host onto the GPU.

kernel<<<>>> takes 2 required and 2 optional parameters.

Required: Number of blocks within the grid and number of threads per block.

Optional: Number of bytes of dynamic shared memory and stream ID.

GPU kernel launches are asynchronous with respect to the host: the host submits the kernel for execution on the GPU, then continues executing subsequent host code without waiting for the kernel to finish.

```
hipMemcpy(d_A, h_A, bytes, hipMemcpyHostToDevice);
hipMemcpy(d_B, h_B, bytes, hipMemcpyHostToDevice);

int thr_per_blk = 256;
int blk_in_grid = (N + thr_per_blk - 1) / thr_per_blk;

vec_add<<<blk_in_grid, thr_per_blk>>>(d_A, d_B, d_C, N);
```

Specific kernel launch.

Generalized form of kernel launch.

```
kernel_name<<< blocks_in_grid,
               threads_per_block,
               [optional] bytes_of_dynamic_shared_memory,
               [optional] stream_id >>>
               (ARG1, ARG2, ...);
```

bytes_of_dynamic_shared_memory and stream_id are 0 by default.

AMD
together we advance_

Public

HIP Error Checking

Check for errors from HIP API calls and kernels.

THEORY

Returns and clears the last recorded HIP Runtime error

Used here to detect immediate kernel launch errors.

`hipGetLastError()`

Blocks the host until all previously submitted GPU work completes

Used here to detect errors that occurred during asynchronous kernel execution.

`hipDeviceSynchronize()`

```
#define HIP_CHECK(cmd) do {                                         \
    hipError_t e = (cmd);                                           \
    if (e != hipSuccess) {                                          \
        fprintf(stderr, "[HIP_CHECK] %s:%d  %s\n",                  \
                __FILE__, __LINE__, hipGetErrorString(e));  \
        exit(EXIT_FAILURE); } } while (0)

int main(){
...
HIP_CHECK( hipMalloc(&d_A, bytes) );
...
vec_add<<<blk_in_grid, thr_per_blk>>>(d_A, d_B, d_C, N);
HIP_CHECK( hipGetLastError() );
HIP_CHECK( hipDeviceSynchronize() );
...
}
```

HIP runtime API calls return a hipError_t status that can be checked when the call returns.

Kernel launches do not return an error directly.

- *Kernel launch errors* can be checked immediately.
- *Kernel execution errors* can occur at any point while the kernel is running, so synchronization is needed to detect them.

AMD
together we advance_

Public

HIP Events – Timing a Kernel

THEORY

HIP events measure elapsed time between points in a GPU stream.

```
hipEvent_t start, stop;
hipEventCreate(&start);
hipEventCreate(&stop);

// Record start in the kernel's stream
hipEventRecord(start, 0);

vec_add<<<grid, block>>>(d_A, d_B, d_C, N);

// Record stop in the same stream
hipEventRecord(stop, 0);
hipEventSynchronize(stop);

float ms;
hipEventElapsedTime(&ms, start, stop);
printf("vec_add: %.3f ms\n", ms);

hipEventDestroy(start); hipEventDestroy(stop);
```

THE 7-STEP PATTERN

1. Create two events
2. Record start in the kernel's stream
3. Launch kernel (or HIP API call) in that stream
4. Record stop in the same stream
5. Wait for the stop event
6. Read elapsed ms
7. Destroy both events

WHAT IS A HIP STREAM?

An ordered queue of GPU work, such as kernel launches and memory copies.

Operations in one stream are processed in submission order, while different streams may overlap.

Here, 0 means the default stream; the events mark points in that queue.

Error checking omitted for clarity.

AMD
together we advance_

Public

Porting from CUDA to HIP

The HIPIFY tools used to translate CUDA source to HIP.

- HIPIFY: hipify-perl vs. hipify-clang
- Porting CUDA Vector Addition to HIP

AMD
together we advance_

Public

HIPIFY: hipify-perl vs. hipify-clang

THEORY

HIPIFY automates most of the translation from CUDA source to HIP.
Choose the tool based on code complexity and setup.

hipify-perl

Regex-based translation. Easy to run; no CUDA installation required.

HOW IT WORKS
Uses regular-expression rules to replace recognized CUDA APIs and syntax with HIP equivalents.

HANDLES
Most CUDA APIs, types, keywords, include directives, and kernel-launch syntax.

USE WHEN
The code uses common CUDA features that translate cleanly to HIP and minimal setup is preferred.

hipify-clang

Clang-based translation. Better for complex C++; requires CUDA installation.

HOW IT WORKS
Uses Clang to parse CUDA as C++ before translating it to HIP.

HANDLES
Supported CUDA APIs and syntax in code that uses templates, namespaces, and other complex constructs.

USE WHEN
The code uses complex C++ and a CUDA installation is available.

After translation, review, build, test, and tune the HIP code.

AMD
together we advance_

Public

Porting CUDA Vector Addition to HIP

PRACTICE

HIPIFY converts CUDA header and runtime API to HIP; kernel source and launch syntax remain unchanged.

CUDA

```cuda
#include <cuda_runtime.h>

__global__ void vec_add(const float* A, const float* B, float* C, int N) {
    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) C[i] = A[i] + B[i];
}

// Abbreviated host code
int main() {
    ...
    cudaMalloc(&d_A, bytes);
    cudaMalloc(&d_B, bytes);
    cudaMalloc(&d_C, bytes);
    ...
    cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, bytes, cudaMemcpyHostToDevice);
    ...
    vec_add<<<blk_in_grid, thr_per_blk>>>(d_A, d_B, d_C, N);
    cudaMemcpy(h_C, d_C, bytes, cudaMemcpyDeviceToHost);
    ...
    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
}
```

HIP (hipify-perl output)

```hip
#include <hip/hip_runtime.h>

__global__ void vec_add(const float* A, const float* B, float* C, int N) {
    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) C[i] = A[i] + B[i];
}

// Abbreviated host code
int main() {
    ...
    hipMalloc(&d_A, bytes);
    hipMalloc(&d_B, bytes);
    hipMalloc(&d_C, bytes);
    ...
    hipMemcpy(d_A, h_A, bytes, hipMemcpyHostToDevice);
    hipMemcpy(d_B, h_B, bytes, hipMemcpyHostToDevice);
    ...
    vec_add<<<blk_in_grid, thr_per_blk>>>(d_A, d_B, d_C, N);
    hipMemcpy(h_C, d_C, bytes, hipMemcpyDeviceToHost);
    ...
    hipFree(d_A); hipFree(d_B); hipFree(d_C);
}
```

```
$ hipify-perl -examine vec_add.cu          # Report translation statistics without writing output
$ hipify-perl vec_add.cu -o vec_add.hip    # Translate the CUDA source to HIP
$ hipcc -o vec_add vec_add.hip && ./vec_add # Build and run the HIP program
```

After translation, review, build, test, and tune the HIP code.

AMD
together we advance_

Public

Module 4 – Summary

Key takeaways from HIP programming and porting.



HIP Programming Concepts

HIP separates host and device work.

The host manages memory and launches kernels; the GPU executes threads organized into grids, blocks, and wavefronts.



HIP Programming

Vector addition shows the HIP runtime workflow.

Allocate memory, copy data, launch kernels, check errors, time with events, verify results, and clean up.



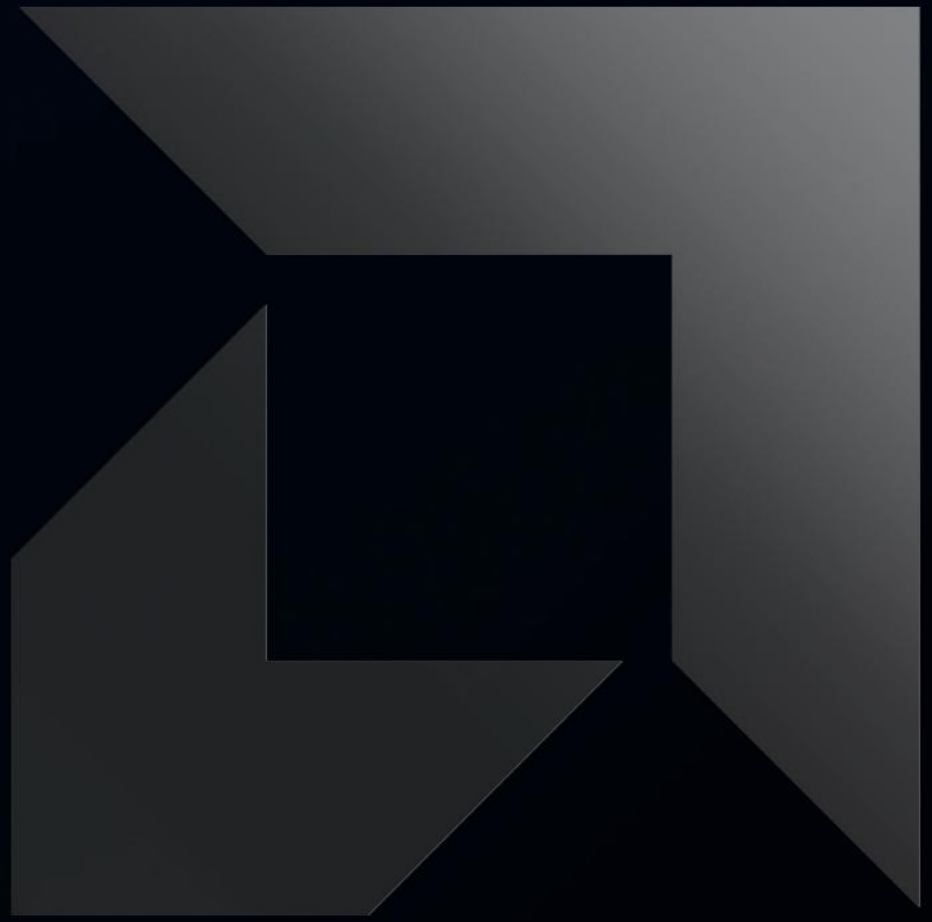
Porting from CUDA to HIP

HIPIFY accelerates CUDA-to-HIP porting.

Use hipify-perl or hipify-clang to translate common CUDA APIs, then review, build, test, and tune the HIP code.

→ NEXT: Module 5 – Profiling and Performance Optimizations

AMD
together we advance_



ROCm™ Certification – Level 1

An Introduction to AMD GPUs and ROCm for AI and HPC

Module 5: Profiling and Performance Optimizations

AMD
together we advance_

Public

Module 5 – Objectives

What you will learn in this module

THEORY



Diagnose Bottlenecks

Roofline + rocprofv3

Use roofline analysis to distinguish memory- from compute-limited kernels, then use rocprofv3 to measure kernel runtime, collect performance counters.



Optimize Memory Access

Coalesce and tile

Use LDS tiling so the transpose performs coalesced global reads and writes – same work, more efficient memory access.



Tune Block Size

Block (workgroup) size

Sweep SAXPY block sizes and observe how available parallelism and resource use affect performance.



Evaluate Reduced Precision

FP16/BF16

Benchmark FP32, FP16, BF16 GEMMs; compare throughput and numerical difference relative to FP32, and explain the role of FP32 accumulation.

AMD
together we advance_

Public

Diagnose Bottlenecks

Before optimizing, identify what limits performance. We will show how roofline analysis and profiling help distinguish memory, compute, and overhead bottlenecks.

- The Roofline Model
- Three Common GPU Bottlenecks
- Profiling – Ask a Question, Then Choose Metrics
- How to Improve Performance

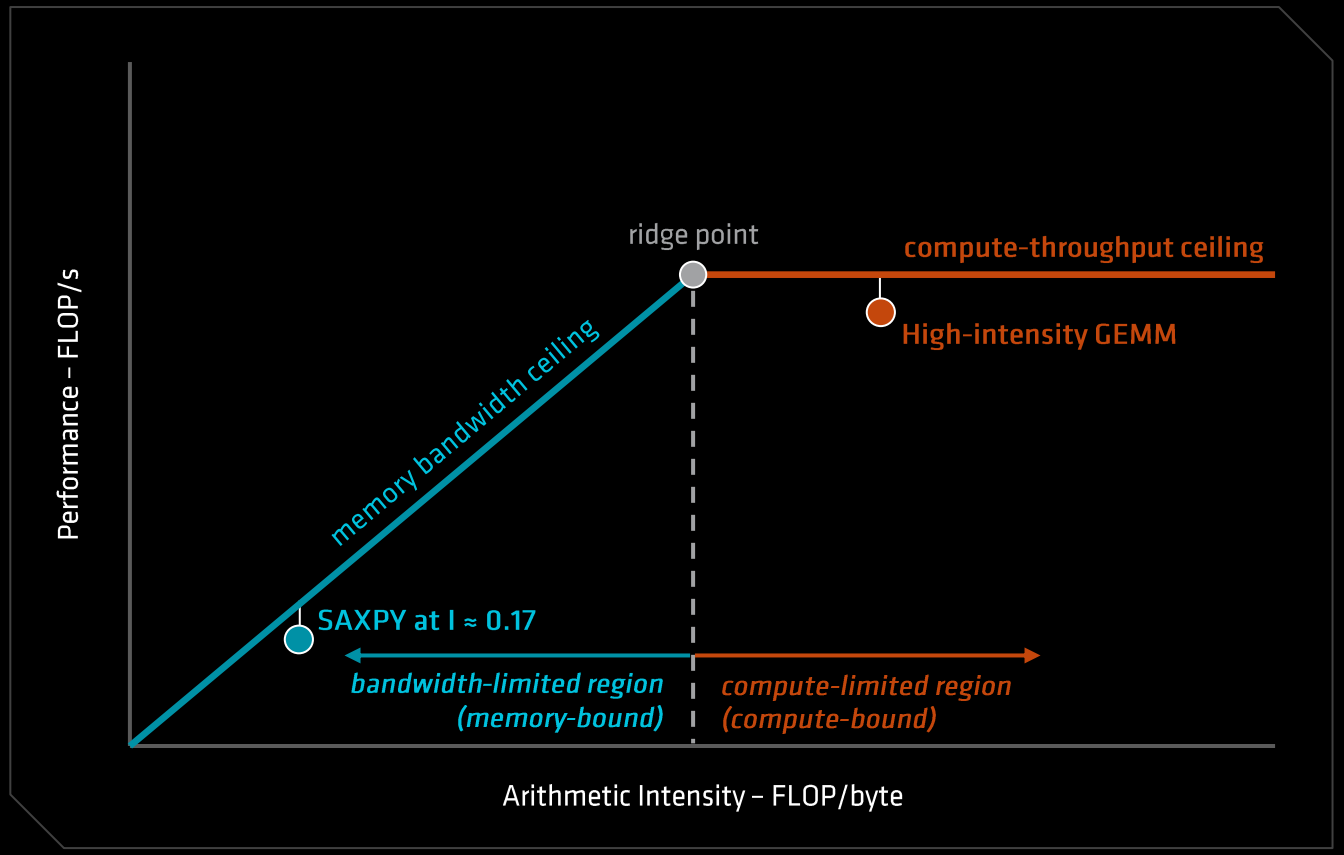
AMD
together we advance_

Public

The Roofline Model

THEORY

Does HBM bandwidth or compute throughput set the kernel's performance ceiling? Arithmetic intensity places the kernel on the roofline.



Arithmetic intensity is the floating-point work performed by a kernel divided by bytes transferred between HBM and the GPU.

Example: FP32 SAXPY (y = a·x + y)

Floating-point operations: 1 multiply (a·x) + 1 add (+y) = 2 FLOPs

Theoretical HBM traffic: read x + read y + write y = 12 bytes

Theoretical arithmetic intensity: 2 FLOP / 12 bytes ≈ 0.17 FLOP/byte

Left of the ridge point, so FP32 SAXPY is memory-bound

LEFT OF RIDGE – memory-bound

Peak or attainable memory (e.g., HBM) bandwidth sets the performance ceiling.

RIGHT OF RIDGE – compute-bound

Peak or attainable compute throughput sets the performance ceiling.

Compute roof: maximum throughput for the datatype and instruction path shown.
Memory roof: B×I, where B is the selected HBM-bandwidth ceiling.
Simplified HBM-level model; measured performance may fall below either roof.

AMD
together we advance_

Public

Three Common GPU Bottlenecks

Most tuning starts by identifying what dominates elapsed time: HBM bandwidth, compute throughput, or launch and synchronization overhead.

MEMORY-BOUND

HBM bandwidth sets the performance ceiling.

ROOFLINE VIEW

Left of the ridge; memory bandwidth sets the ceiling.

TYPICAL OPTIMIZATIONS

Move fewer bytes, coalesce accesses, and reuse data.

COMPUTE-BOUND

Compute throughput sets the performance ceiling.

ROOFLINE VIEW

Right of the ridge; compute throughput sets the ceiling.

TYPICAL OPTIMIZATIONS

Use optimized libraries or instructions, reduce unnecessary work, or use lower precision where accuracy permits.

OVERHEAD-DOMINATED

Kernel launches, synchronization, or too little work per launch dominate elapsed time.

ROOFLINE VIEW

May sit far below the roofline; roofline position alone does not identify the cause.

TYPICAL OPTIMIZATIONS

Batch or fuse small operations, reduce launches and synchronization, and increase useful work per launch.

AMD
together we advance_

Public

Profiling – Ask a Question, Then Choose Metrics

Start with kernel timing to find where time is spent. Collect selected counters to test a compute- or memory-bottleneck hypothesis.

```
# First pass: identify important kernels
$ rocprofv3 --kernel-trace --stats --output-format csv -- ./app

# Second pass: collect a focused counter set
$ rocprofv3 --pmc <selected counters> --output-format csv -- ./app
```

Generates kernel traces, summary statistics, and selected counter data.

Use rocprofv3-avail list --pmc to see counters supported by the GPU.

KERNEL TIMING

AverageNs

Average time per launch.

Calls

Number of launches.

TotalDurationNs / Percentage

Contribution to total kernel time.

Start here:
which kernels consume meaningful total time?

COMPUTE EVIDENCE

Wavefront activity

How much parallel work was dispatched.

Vector / matrix instruction activity

How much arithmetic work was executed.

Active-lane utilization

How fully the wavefront lanes participated.

Compute clue:
high arithmetic activity with modest memory pressure.

MEMORY EVIDENCE

Bytes read / written

How much memory traffic was generated.

Memory-unit activity

How heavily the memory path was used.

Memory stalls / wait cycles

How often data access delayed progress.

Memory clue:
high traffic or stalls relative to compute activity.

No single counter proves the bottleneck; interpret timing and several related metrics together.

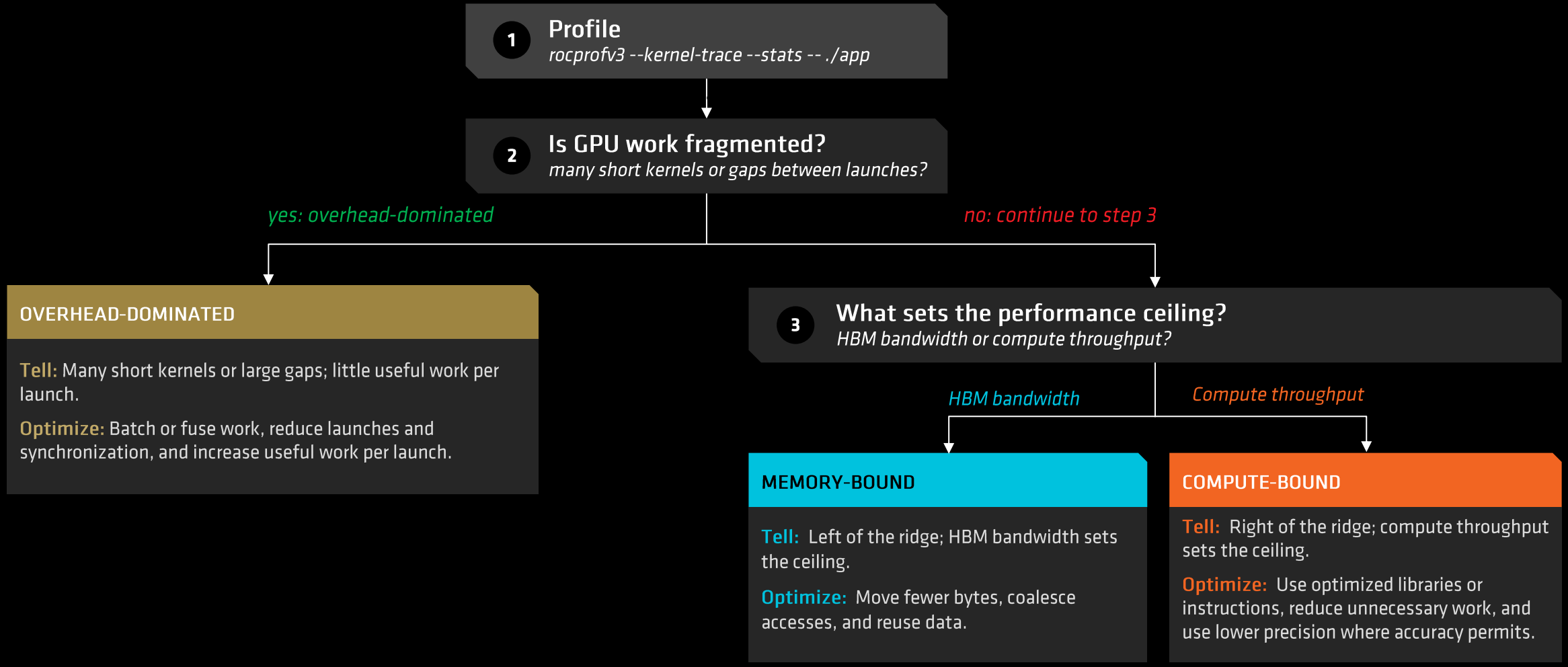
AMD
together we advance_

Public

How to Improve Performance

THEORY

Use profiling and roofline analysis to identify the dominant limit, then choose an optimization that targets it.



AMD
together we advance_

Public

Three Optimization Strategies

Each example targets a different performance limit.

01

Memory access efficiency

Matrix transpose: strided vs. coalesced global access and LDS tiling.

02

Workgroup size and parallelism

SAXPY: sweep block size and observe how available wavefronts affect performance.

03

Compute throughput and precision

PyTorch GEMM: FP32 vs. FP16/BF16, matrix acceleration, and the throughput–accuracy tradeoff.

AMD
together we advance_

Public

Optimize Memory Access

We will use matrix transpose to show how strided access reduces memory efficiency and how coalescing and LDS tiling address it.

- Matrix Transpose
- Mapping a 2D Matrix to 1D Memory
- Mapping a 2D HIP Grid onto the Matrix
- Memory Coalescing
- Shared Memory (LDS) Staging
- Baseline to Optimized Kernel

AMD
together we advance_

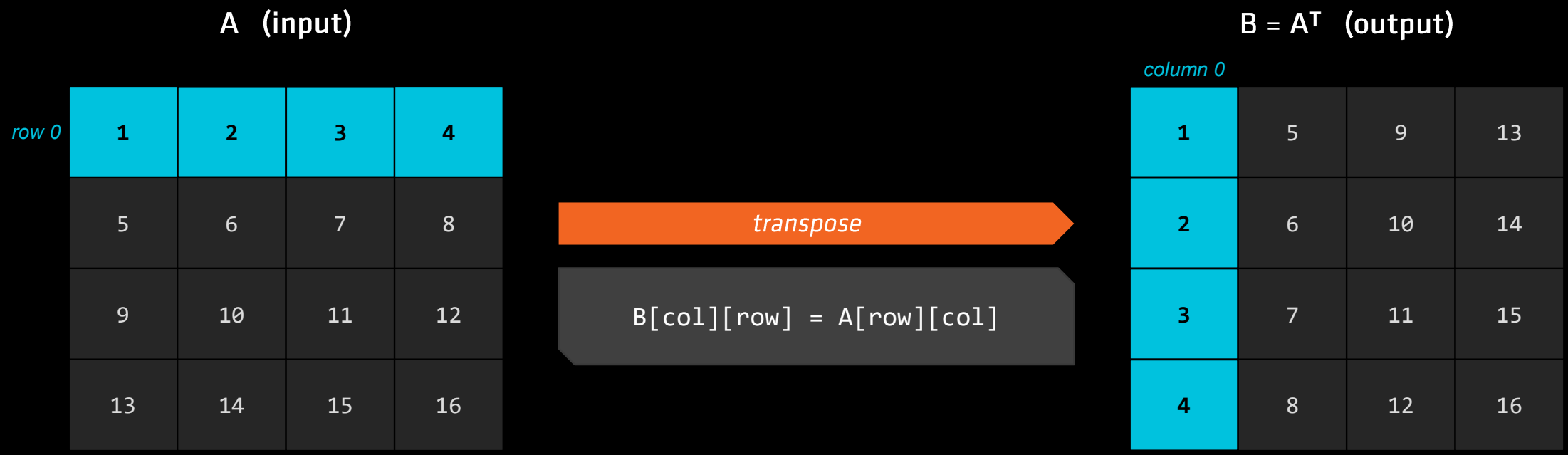
Public

Matrix Transpose

THEORY

Transpose swaps row and column indices: an M × N matrix becomes N × M, and A[row][col] moves to B[col][row].

bottleneck STRIDED GLOBAL-MEMORY WRITES



The challenge: Adjacent threads read contiguous elements from an input row but write strided elements to an output column. This access mismatch is the main optimization challenge.

AMD
together we advance_

Public

Mapping a 2D Matrix to 1D Memory

THEORY

A 7 × 7 logical matrix (A[row][col]) is stored in row-major order as one linear array (A[index]).

columns →

rows ↓

A[0][0]	A[0][1]	A[0][2]	A[0][3]	A[0][4]	A[0][5]	A[0][6]
A[1][0]	A[1][1]	A[1][2]	A[1][3]	A[1][4]	A[1][5]	A[1][6]
A[2][0]	A[2][1]	A[2][2]	A[2][3]	A[2][4]	A[2][5]	A[2][6]
A[3][0]	A[3][1]	A[3][2]	A[3][3]	A[3][4]	A[3][5]	A[3][6]
A[4][0]	A[4][1]	A[4][2]	A[4][3]	A[4][4]	A[4][5]	A[4][6]
A[5][0]	A[5][1]	A[5][2]	A[5][3]	A[5][4]	A[5][5]	A[5][6]
A[6][0]	A[6][1]	A[6][2]	A[6][3]	A[6][4]	A[6][5]	A[6][6]

```
index = row * width + col

(width = 7 in this example)
```

convert 2D coordinates to a linear index →

row	col	calculation	index
0	0	7 * 0 + 0	0
0	3	7 * 0 + 3	3
2	6	7 * 2 + 6	20
5	0	7 * 5 + 0	35
6	6	7 * 6 + 6	48

LINEAR (1D) STORAGE — wrapped into rows for readability

A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	A[6]
A[7]	A[8]	A[9]	A[10]	A[11]	A[12]	A[13]
A[14]	A[15]	A[16]	A[17]	A[18]	A[19]	A[20]
A[21]	A[22]	A[23]	A[24]	A[25]	A[26]	A[27]
A[28]	A[29]	A[30]	A[31]	A[32]	A[33]	A[34]
A[35]	A[36]	A[37]	A[38]	A[39]	A[40]	A[41]
A[42]	A[43]	A[44]	A[45]	A[46]	A[47]	A[48]

Why this matters: Elements in the same row are contiguous in memory, while elements in the same column are one row stride apart.

AMD
together we advance_

Public

Mapping a 2D HIP Grid onto the Matrix

THEORY

A 2 × 2 grid of 4 × 4-thread blocks launches an 8 × 8 thread space to cover the 7 × 7 matrix.



GRID	2 × 2 blocks
BLOCK	4 × 4 threads
THREAD SPACE	8 × 8
TOTAL	64 threads (4 × 16)
MATRIX	49 elements (7 × 7)
OOB	15 threads

Red boundaries outline 4 × 4 thread blocks

Gray threads outside matrix boundary (OOB).

KERNEL LAUNCH CONFIGURATION

```
dim3 thr_per_blk(4, 4);
dim3 blk_in_grid(
     (width  + thr_per_blk.x - 1) / thr_per_blk.x),
     (height + thr_per_blk.y - 1) / thr_per_blk.y));
```

For this example, width = height = 7, so blk_in_grid(2, 2)

GLOBAL THREAD COORDINATES

```
int col  = blockIdx.x * blockDim.x + threadIdx.x;
int row  = blockIdx.y * blockDim.y + threadIdx.y;
```

BOUNDS CHECK & LINEAR (1D) INDEX

```
if (row < height && col < width) {
    int index = row * width + col;
    // Process A[index]
}
```

A SPECIFIC THREAD MAPPING EXAMPLE

```
blockIdx = (1, 1); threadIdx = (2, 1)

col   = 1 * 4 + 2 = 6;
row   = 1 * 4 + 1 = 5;
index = 5 * 7 + 6 = 41;
```

This thread maps to logical element A[5][6], stored at linear location A[41]

Each thread computes one global (row, col) coordinate; gray edge threads fail the bounds check and skip the memory access.

AMD
together we advance_

Public

Matrix Transpose: Baseline Kernel

PRACTICE

Each thread reads one input element and writes it directly to the transposed output—one element per thread, with no shared memory (LDS).

```
__global__ void transpose_baseline(const float* in,
                                   float* out,
                                   int width,
                                   int height) {
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int row = blockIdx.y * blockDim.y + threadIdx.y;

    if (col < width && row < height) {
        out[col * height + row] = in[row * width + col];

    // 1. Consecutive col values access consecutive input elements
    // 2. Consecutive col values access output elements height apart

    }
}
```

WHAT THE CODE DOES

Each thread reads one input element from global memory and writes it directly to its transposed output location.

1 CONTIGUOUS INPUT READS

For threads with the same row and consecutive col values, in[row*width+col] advances by one element.

`in[row*width+0], in[row*width+1], in[row*width+2], …`

2 STRIDED OUTPUT WRITES

For those same threads, out[col*height+row] advances by height elements.

`out[0*height+row], out[1*height+row], out[2*height+row], …`

Optimization goal: *Keep one global load and one global store per element, but reorganize the global accesses so neighboring threads use neighboring addresses.*

Next: Memory coalescing and LDS staging.

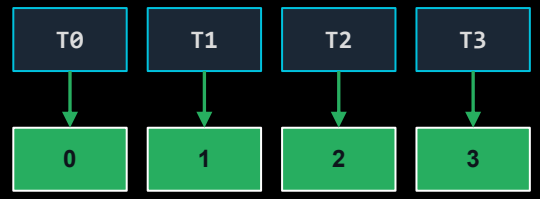
AMD
together we advance_

Public

Memory Coalescing

Neighboring memory accesses combine into fewer transactions; strided accesses require more.

COALESCED — fewer memory transactions



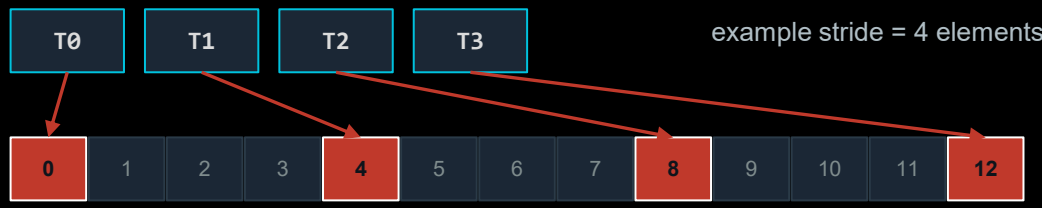
TN = one thread (a lane in a wavefront)

□ OR □ = 1 requested global-memory element

consecutive lanes → consecutive addresses → **fewer, wider transactions**

- Thread 0 → addr[k]
- Thread 1 → addr[k + 1]
- Thread 2 → addr[k + 2] ...
- Neighboring lanes access neighboring addresses
- Requests combine into wide, aligned transactions
- High transaction efficiency

STRIDED — more memory transactions



consecutive lanes → strided addresses → **many transactions**

- Thread 0 → addr[k]
- Thread 1 → addr[k + stride]
- Thread 2 → addr[k + 2 × stride] ...
- Neighboring lanes access widely separated addresses
- Requests span many memory regions
- Lower transaction efficiency and effective bandwidth

4 lanes shown from a 64-thread wavefront.

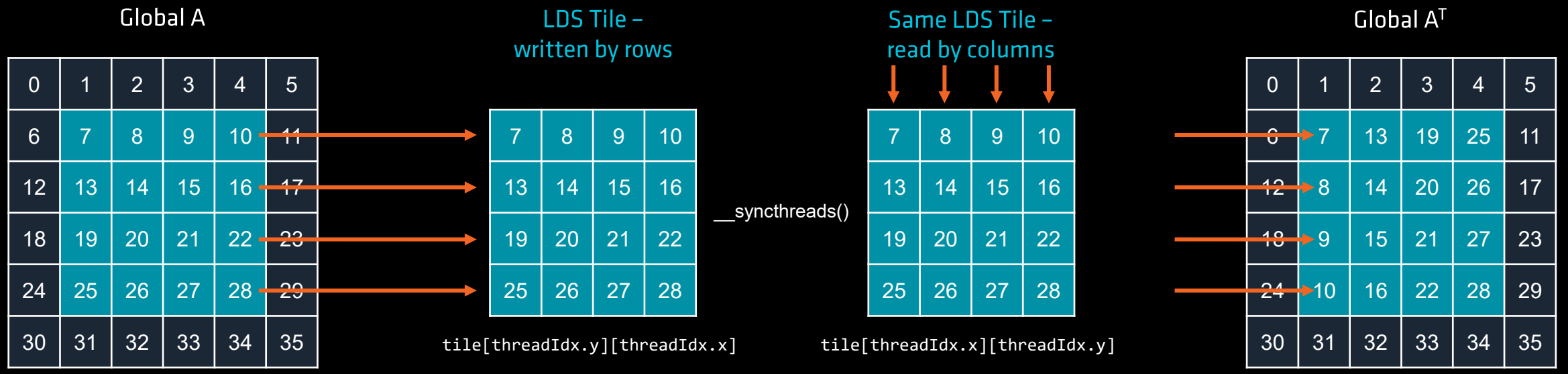
AMD
together we advance_

Public

Shared Memory (LDS) Staging

THEORY

Load each tile into LDS by rows, synchronize, then read with swapped indices and write the transpose by rows.



Threads cooperatively load the input tile row-wise into LDS, then synchronize before reading it back.

Then threads read the same LDS tile with swapped indices and write the transposed tile row-wise to global memory.

```
__shared__ float tile[TILE][TILE];
   int in_col = blockIdx.x * TILE + threadIdx.x;
   int in_row = blockIdx.y * TILE + threadIdx.y;
   if (in_col < width && in_row < height)
      tile[threadIdx.y][threadIdx.x] =
           in[in_row * width + in_col];  // coalesced read
   __syncthreads();
```

```
   int out_col = blockIdx.y * TILE + threadIdx.x;
   int out_row = blockIdx.x * TILE + threadIdx.y;
   if (out_col < height && out_row < width)
      out[out_row * height + out_col] =
          tile[threadIdx.x][threadIdx.y];  // coalesced write
```


AMD
together we advance_

Public

Matrix Transpose: Optimized Kernel

PRACTICE

Stage each tile in LDS, synchronize, then read with swapped indices—coalesced global loads and stores.

```
#define TILE 16
__global__ void transpose_optimized(const float* in, float* out, int width, int height) {
    __shared__ float tile[TILE][TILE + 1]; // +1 padding reduces LDS bank conflicts

    // Input coordinates in the original height × width matrix
    int in_col = blockIdx.x * TILE + threadIdx.x;
    int in_row = blockIdx.y * TILE + threadIdx.y;

    // Load the input tile row-wise into LDS
    if (in_col < width && in_row < height) {
        tile[threadIdx.y][threadIdx.x] = in[in_row * width + in_col];
    }

    __syncthreads();  // The entire workgroup must finish loading the tile

    // Swap the block coordinates for the transposed output
    int out_col = blockIdx.y * TILE + threadIdx.x;
    int out_row = blockIdx.x * TILE + threadIdx.y;

    // Output has width rows × height columns
    if (out_col < height && out_row < width) {
        out[out_row * height + out_col] = tile[threadIdx.x][threadIdx.y];
    }
}
```

WHAT CHANGED FROM BASELINE

- **+1 padding:** reduces LDS bank conflicts during transposed reads.
- **Coalesced load:** Neighboring threads read neighboring global-memory addresses into an LDS row.
- **Synchronize:** Wait until the tile is fully loaded before any thread reads values written by another thread.
- **Swapped LDS read + row-wise store:** The baseline's strided write becomes a coalesced global store.

`__shared__` allocates one tile per workgroup in on-chip LDS. Ordinary assignments move each value between global memory and LDS—no separate copy API is required.

Launch configuration: `dim3 thr_per_blk(TILE, TILE); // 16 × 16 = 256 threads`

AMD
together we advance_

Public

Tune Block Size

We will use SAXPY to show how block size affects wavefront organization and what to consider when choosing a launch configuration.

- SAXPY
- SAXPY: Baseline Kernel
- Does SAXPY Reach the Bandwidth Ceiling?
- SAXPY: Sweep the Block Size

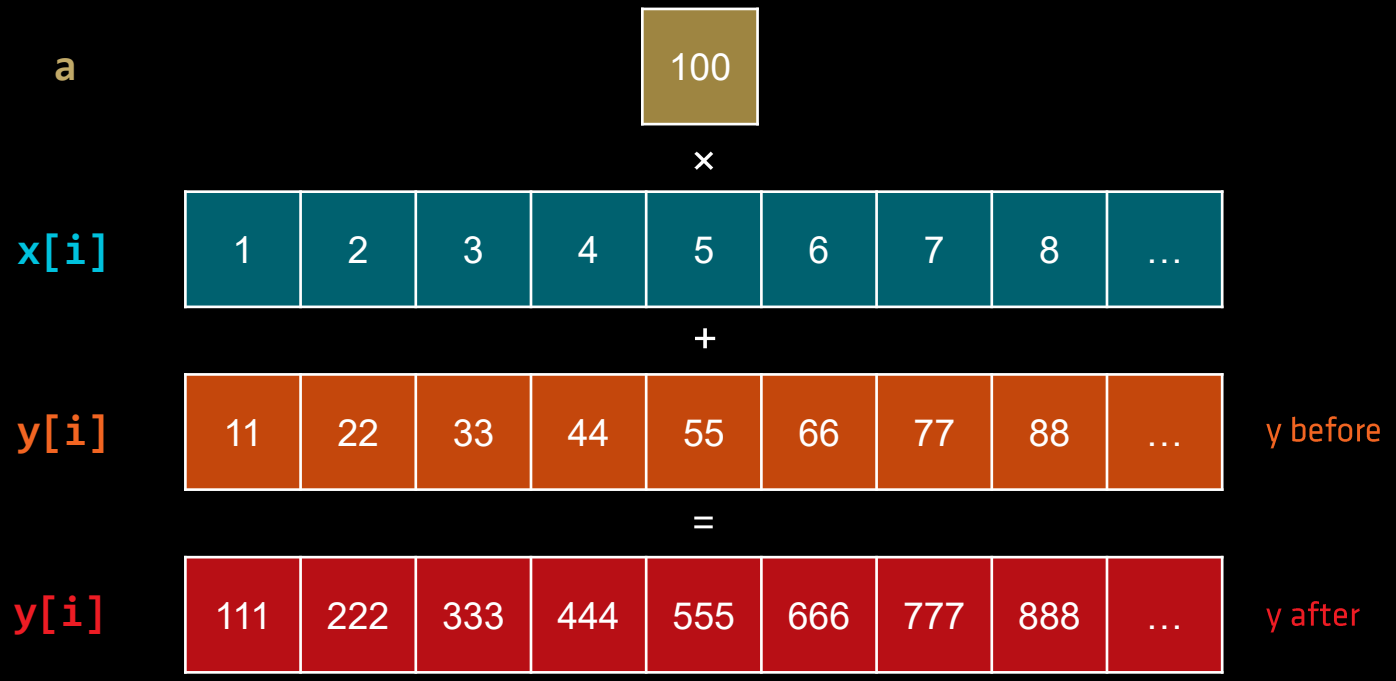
AMD
together we advance_

Public

SAXPY

Single-precision A·X Plus Y

THEORY



SAXPY

$y[i] = a \cdot x[i] + y[i]$

- **PERFORMANCE LIMIT:** MEMORY BANDWIDTH
- **QUESTION:** HOW CLOSE ARE WE TO THE CEILING?

The challenge: At only I≈0.17 FLOP/byte, FP32 SAXPY is bandwidth-bound. The optimization task is not to make the arithmetic faster, but to determine whether the kernel approaches the available memory-bandwidth ceiling.

AMD
together we advance_

Public

SAXPY: Baseline Kernel

One element per thread with contiguous memory access.

PRACTICE

```
// SAXPY: y = a*x + y   (memory-bound)
__global__ void saxpy(float a, const float* x, float* y, int N) {

    int i = blockDim.x * blockIdx.x + threadIdx.x; // 1 elem/thread
    if (i < N) y[i] = a*x[i] + y[i];               // 2 reads, 1 write
}
```

```
// Kernel launch configuration
int thr_per_blk = 256;
int blk_in_grid = (N + thr_per_blk - 1) / thr_per_blk;

saxpy<<<blk_in_grid, thr_per_blk>>>(a, d_x, d_y, N);
```

WHAT TO NOTICE

INDEPENDENT WORK

One element per thread – no communication or synchronization between threads.

EFFICIENT ACCESS PATTERN

Neighboring threads access neighboring elements – the loads and stores are naturally coalesced.

LOW ARITHMETIC INTENSITY

Two loads + one store, only two FLOPs – data movement dominates the work.

AMD
together we advance_

Public

Does SAXPY Reach the Bandwidth Ceiling?

PRACTICE

Use profiling data to test the roofline prediction.

How close is measured SAXPY bandwidth to the memory bandwidth ceiling?

Memory-bound performance ceiling for SAXPY:
$P_{memory\ roof}$ = (5.3 TB/s) * 0.17 FLOP/B ≈ 0.88 TFLOP/s

Run SAXPY kernel in warm-up loop, then in timed loop for multiple iterations to measure avg time for one kernel execution.

1. Measure average time (ns) to run SAXPY size N.

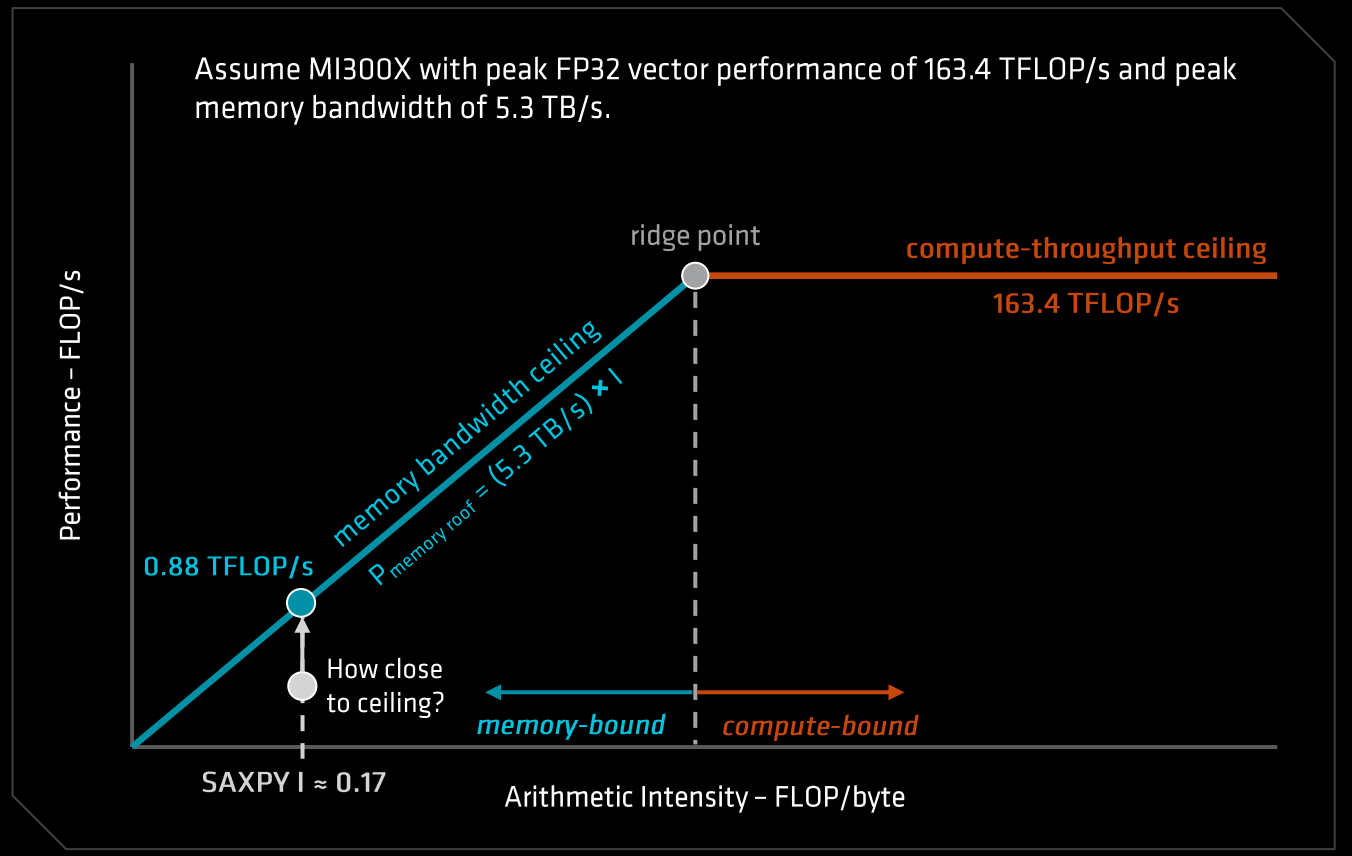
```
$ rocprofv3 --kernel-trace --stats \
  --output-format csv -- ./saxpy
```

2. Calculate effective bandwidth (GB/s):

$$\text{Effective Bandwidth} = \frac{12N}{t_{avg,\ ns}}$$

3. Calculate $P_{measured} = B_{effective} * I_{SAXPY}$

$P_{measured} = B_{effective} * I_{SAXPY} = B_{effective} * 0.17$



Plot (I_{SAXPY}, $P_{measured}$) ⟶ Fastest implementation is the point closest to ceiling.

Numbers shown here are peak theoretical; measured points should be compared against measured sustainable bandwidth for a more realistic attainable ceiling.

AMD
together we advance_

Public

SAXPY: Sweep the Block Size

Change one launch parameter while keeping the workload fixed.

PRACTICE

Launch configuration: sweep block sizes

```
int thr_per_blk = ...;       // 64, 128, 256, 512
int blk_in_grid = (N + thr_per_blk - 1) / thr_per_blk;

saxpy<<<blk_in_grid, thr_per_blk>>>(a, d_x, d_y, N);
```

RECORD THE BENCHMARK RESULTS

Threads/block	Avg. kernel time (ns)	Effective bandwidth	Bandwidth utilization (%)
64	-	-	-
128	-	-	-
256	-	-	-
512	-	-	-

All block sizes remain at I≈0.17 FLOP/byte. Sweep the block size to find the point where bandwidth plateaus; beyond that, larger blocks provide little benefit.

EXPERIMENT PROCEDURE

1 KEEP COMPARISON CONTROLLED

Use the same N, inputs, warm-up count, and number of timed launches.

2 TEST SEVERAL BLOCK SIZES

Run 64, 128, 256, and 512 threads per block as separate profiling runs.

3 RECORD AVG KERNEL TIME

```
rocprofv3 --kernel-trace --stats \
  --output-format csv -- \
  ./saxpy <threads_per_block>
```

4 CALCULATE AND COMPARE

$$B_{effective} = \frac{12N}{t_{avg,\ ns}} \qquad BW_{utilization} = \frac{B_{effective}}{B_{ceiling}}$$

Look for bandwidth plateau.

AMD
together we advance_

Public

Evaluate Reduced Precision

We will use GEMM to show how FP16/BF16 can increase throughput and how to evaluate the numerical tradeoff against FP32.

- Mixed-Precision GEMM – Throughput vs. Accuracy
- GEMM Precision Benchmark (PyTorch)

AMD
together we advance_

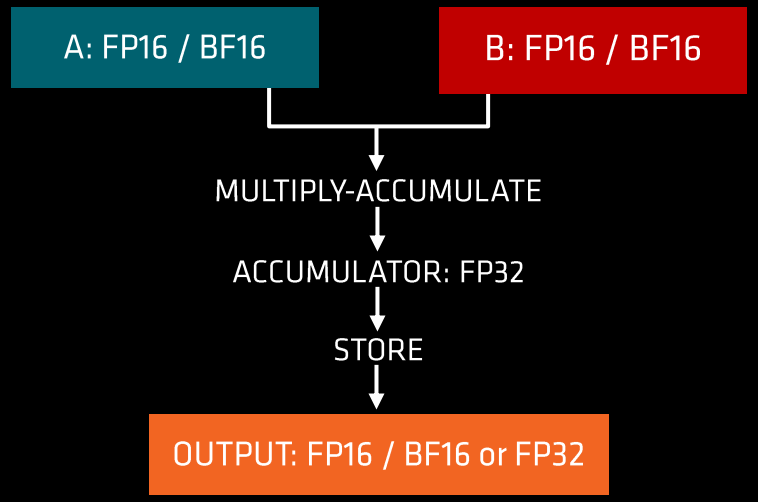
Public

Mixed-Precision GEMM – Throughput vs. Accuracy

THEORY

FP16/BF16 operands can increase GEMM throughput; FP32 accumulation reduces summation error.

COMMON MIXED-PRECISION GEMM PATH



Input, accumulation, output precision are separate choices. Exact behavior depends on API, library, hardware, settings.

EXPECTED LIMIT: COMPUTE THROUGHPUT

Format	Bits	Exponent	Fractional Bits	Decimal Digits
FP32	32	8	23	~7
FP16	16	5	10	~3–4
BF16	16	8	7	~2–3

Each format also includes one sign bit. FP16 provides more precision; BF16 preserves FP32's exponent range.

WHY FASTER

16-bit operands use half the storage of FP32, reducing operand traffic. Eligible GEMMs can also use higher-throughput matrix instructions.

THE TRADEOFF

FP16/BF16 round the input values. FP32 accumulation reduces error as the products are summed, but cannot recover precision already lost in the inputs.

THE EXPERIMENT

Run the same M=N=K=2048 GEMM in FP32, FP16, and BF16. Compare runtime, TFLOP/s, and error relative to the FP32 result.

AMD
together we advance_

Public

GEMM Precision Benchmark (PyTorch)

PRACTICE

Same source matrices, three operand dtypes. Measure throughput and numerical error.

PyTorch benchmark pseudocode; helper implementations omitted

```
device = torch.device("cuda")   # ROCm uses PyTorch's CUDA API
M = N = K = 2048

# One source problem for all three tests
A32 = torch.rand(M, K, device=device)
B32 = torch.rand(K, N, device=device)
C_ref = torch.mm(A32, B32)

for dtype in [torch.float32, torch.float16, torch.bfloat16]:

    A = A32.to(dtype)
    B = B32.to(dtype)

    warm_up(torch.mm, A, B)
    ms = time_gpu(torch.mm, A, B, repeats=50)   # Average ms/GEMM

    C = torch.mm(A, B)   # Untimed result for comparison
    tflops = 2 * M * N * K / (ms * 1e9)
    difference = relative_difference(C.float(), C_ref)

    report(dtype, ms, tflops, difference)
```

time_gpu() returns the average elapsed milliseconds per GEMM using GPU events and synchronization. relative_difference() compares the result with the FP32 reference.

RECORD THE BENCHMARK RESULTS

Operand dtype	Avg. time (ms)	TFLOP/s	Rel. difference vs. FP32
FP32	-	-	reference
FP16	-	-	-
BF16	-	-	-

Choose the fastest precision that stays within the workload's numerical tolerance.

AMD
together we advance_

Public

Module 5 – Summary

THEORY

Diagnose first. Optimize the dominant limit. Measure the result.



Diagnose Bottlenecks

Measure before changing code

Use kernel timing to find important work, then use roofline analysis and profiling evidence to identify the dominant performance limit.



Optimize Memory Access

Same work, better memory access

Use LDS tiling so the transpose performs coalesced global reads and writes – same work, more efficient memory access.



Tune Block Size

Block size is a tuning parameter

The SAXPY sweep showed that block size affects available parallelism and resource use–benchmark several sizes rather than assuming one is best.



Evaluate Reduced Precision

Throughput must still meet accuracy

FP16/BF16 can accelerate GEMM, while FP32 accumulation can reduce summation error. Measure both throughput and numerical difference before choosing a datatype.

AMD
together we advance_